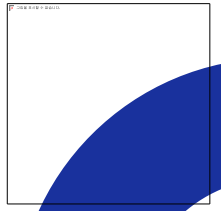
반도체장비예지보전실무 03

데이터 탐색과 모델 개선

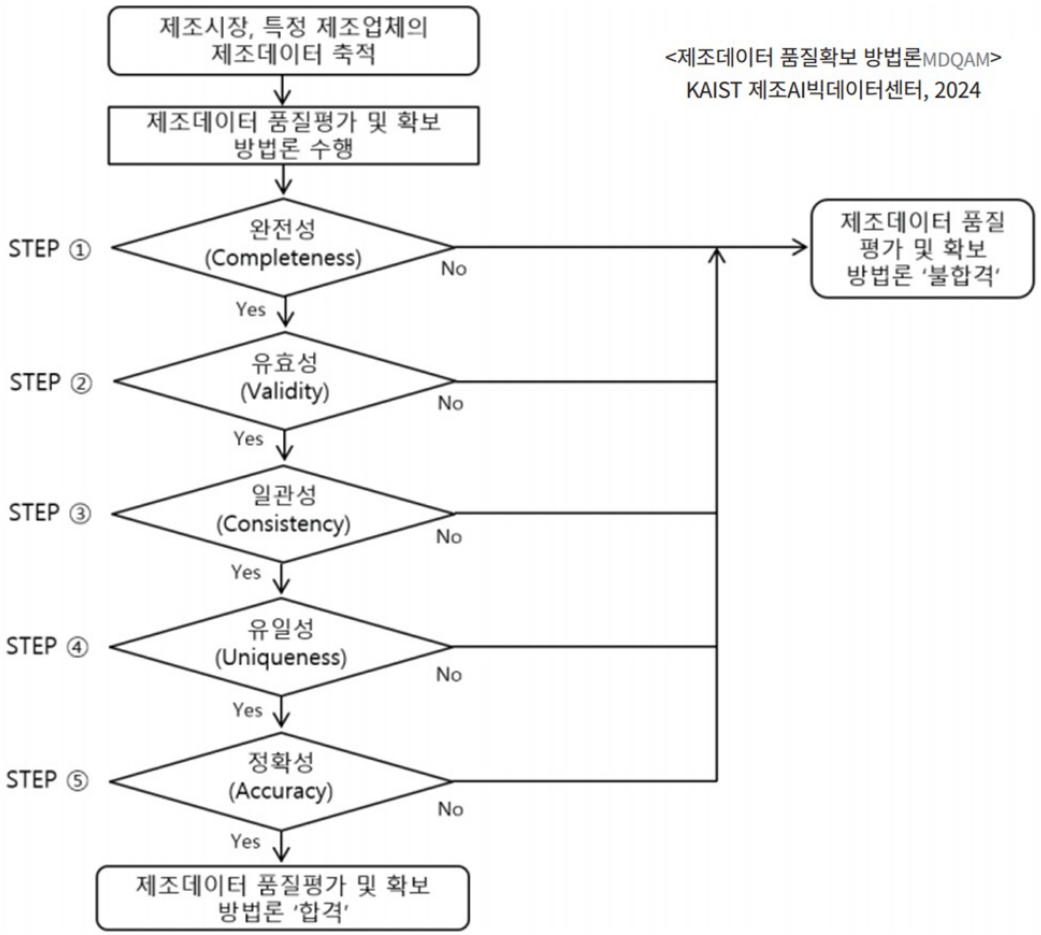


김 민 진 | muea.mj@gmail.com

REVIEW

제조데이터 품질확보 방법론

STEP	품질평가 지표	품질평가 내용	품질평가 지수 불량 제조데이터가 발생할 수 있는 상황
1	완전성 (Completeness)	필수 제조데이터 항목에 누락이 없어야 한다.	q1 = (1-(결측치 수/전체 제조데이터 수))*100
2	유효성 (Validity)	제조데이터는 제조현장의 도메인을 반영한 관리범위 내에 존재해야 한다.	q2 = (유효성 만족 데이터 수/STEP ① 전처리 후 제조데이터 수)*100
3	일관성 (Consistency)	표준 제조데이터 스크립트가 제시하는 통일된 속성규칙을 가져야 한다.	q3 = (일관성 만족 데이터 수/STEP ② 전처리 후 제조데이터 수)*100
4	유일성 (Uniqueness)	제조데이터의 키값은 중복 없이 독립적인 값을 가져야 한다.	q4 = (1-(중복 데이터 수/STEP ③ 전처리 후 제조데이터 수))*100
5	정확성 (Accuracy)	제조데이터는 실제 제조현장의 공정상황을 반영해야 한다.	q5 = (1-(정확성 위배 데이터 수/STEP ④ 전처리 후 제조데이터 수))*100



<제조데이터 품질확보 방법론MDQAM>
KAIST 제조AI빅데이터센터, 2024

표준 제조데이터 스크립트

- 데이터에 대한 설명 (metadata)
  - 이 데이터가 무엇인지 설명하는 데이터 명세서

- 제조데이터가
  - **어떤 목적으로, 누가, 언제,**
  - **어떤 방식으로 수집했는지**와
  - 각 변수의 **이름, 의미, 척도,**
  - **수집 방법, 수집 주기** 등을 정리한
  - **데이터 정의·명세 문서**

〈표준 제조데이터 스크립트(예)〉

(파일형식) 제공기업/기관명_데이터셋명_기준일자.hwp

파일명 : ㈜한국플라스틱_사출성형기_제조AI데이터셋_종합설명_20230911.hwp

기업명 : K-플라텍 | **업종 :** 사출성형 (플라스틱)

공장 : 울산 제1공장 | **공정 :** 플라스틱사출 | **설비명 :** LS엠트론 (WIZ-E시리즈)

제조데이터 수집목적 : ① 제조데이터를 통한 생산이력 기록 ② AI분석을 통한 불량원인진단 분석

제조데이터 수집방법 :
사출압력(sensor), 사출시간(PLC), 사출속도 (PLC), 냉각수온도(sensor), 호퍼온도(sensor), 품질(inspection)

[제조데이터 표준 스크립트(표준사전)] ID, 변수명, 척도, 형태, 수집방법, 수집주기, 변수(속성)명, 범주값

헤더			바디					
ID	공정 변수명 (한글)	변수명 (프로그램)	척도	형태	수집방법 (센서유형)	수집 주기	정의 / 내용	범주 값
A1	사출 압력	Injection_pressure	연속형	float64	설비 WIZ-E의 압력센서	1sec	설정된 속도에 따라 금형에서 발생하는 압력에 대한 제한 압력	-
A2	사출 시간	Injection_time	연속형	datetime	PLC 사출시간	1sec	공급 시간 및 유지 시간이 포함	-
A3	사출 속도	Injection_speed	연속형	float64	PLC 사출속도	1sec	사출기의 노즐에서 사출되는 수지의 속도	-
A4	냉각수 온도	Water_temperature	연속형	float64	설비 WIZ-E의 온도센서	1sec	냉각수의 정상적인 온도는 80~90℃	-
A5	호퍼 온도	Hopper_temperature	연속형	float64	설비 WIZ-E의 온도센서	1sec	원재료를 한 군데로 모아서 이송시켜주는 V자형 도구의 온도	-
A6	품질	quality	명목형	int64	육안 검사 결과	5min	제조물의 품질결정, 모델의 종속변수	'0' (양품) '1' (불량)

1. 탐색적 데이터 분석의 이해

탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 개념
  - 본격적으로 머신러닝 모델을 만들기 전에 **데이터의 특징과 상태를 살펴보는 과정**
  - 데이터를 자유롭게 탐색하면서 **패턴, 문제점, 가설을 발견하는 것이 목적**

- 표, 통계값, 그래프 등을 통해 다음과 같은 내용을 파악함
  - **데이터가 어떤 값들로 구성되어 있는가**
  - **값의 범위와 분포는 어떠한가**
  - **결측치나 이상치가 있는가**
  - **변수들 사이에 어떤 관계가 있는가**
  - **분석이나 모델링 전에 어떤 전처리가 필요한가**

탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 기초 기술 분석

DataFrame 함수	무엇을 확인하나?	결과에서 파악할 것
df.head() / df.sample()	데이터의 실제 모습	• 어떤 컬럼들이 있는가? • 각 컬럼에는 어떤 값이 들어 있는가? • 수치형 데이터인가, 범주형 데이터인가? • 값의 단위나 의미를 추측할 수 있는가? • 비어 있거나 이상해 보이는 값은 없는가?
df.info()	데이터의 구조와 결측치	• **데이터 크기** : **몇 개의 행과 컬럼**이 있는지 확인 • Non-Null Count : 각 컬럼에서 값이 존재하는 **행의 수 확인** → 전체 행 수보다 작으면 결측치가 있다는 뜻 • **컬럼 이름** : 분석에 필요한 **변수가 어떤 것**인지 확인 • **각 컬럼의 자료형** : 전처리가 필요한 컬럼이 있는지 확인
df.describe()	수치형 데이터의 통계적 특징	• 평균, 표준편차, 최솟값·최댓값, 사분위수를 통해 값의 범위와 분포를 대략 확인

탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 기초 기술 분석: 예시

```
df.sample(5)
```

	Lot	Temp	Viscosity	Failure	FailureCode
796	2110904363	25.5600	1.1500	0	None
837	2110826334	NaN	0.9700	0	None
646	2109083909	27.0200	1.7000	0	None
411	2110098221	26.3400	2.5300	0	None
448	2110198772	24.3700	2.4000	0	None

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Lot          1000 non-null   int64
 1   Temp         1000 non-null   float64
 2   Viscosity    1000 non-null   float64
 3   Failure      1000 non-null   object
 4   FailureCode  1000 non-null   object
dtypes: float64(2), int64(1), object(2)
memory usage: 39.2+ KB
```

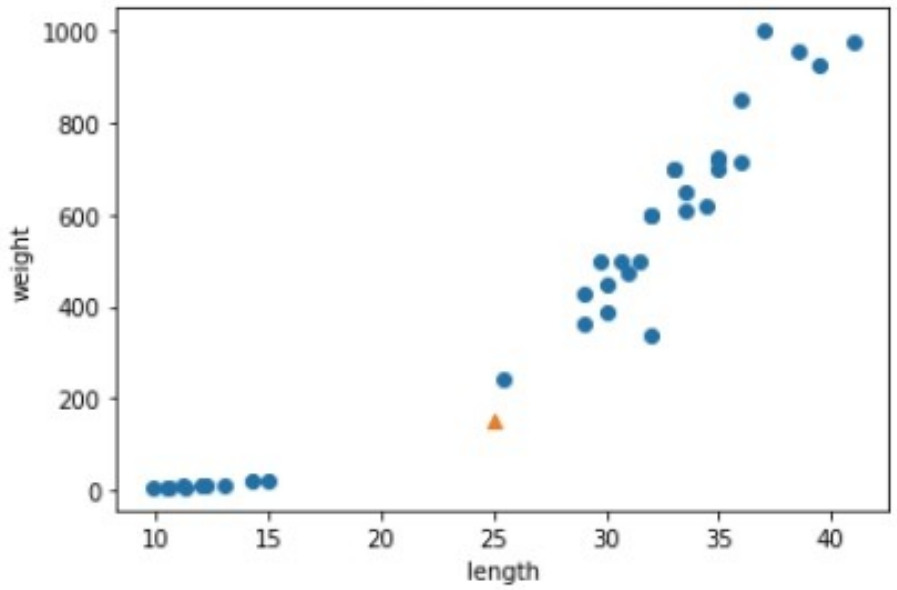
```
df.describe()
```

	Lot	Temp	Viscosity
count	1000.0000	1000.0000	1000.0000
mean	2110175932.5680	30.6861	5.2547
std	677478.9190	30.8901	11.3825
min	2109021818.0000	2.4500	0.7400
25%	2109585209.0000	24.5800	1.6475
50%	2110189868.0000	25.4300	2.3900
75%	2110782208.0000	26.3025	3.1400
max	2111289797.0000	435.3500	101.8000

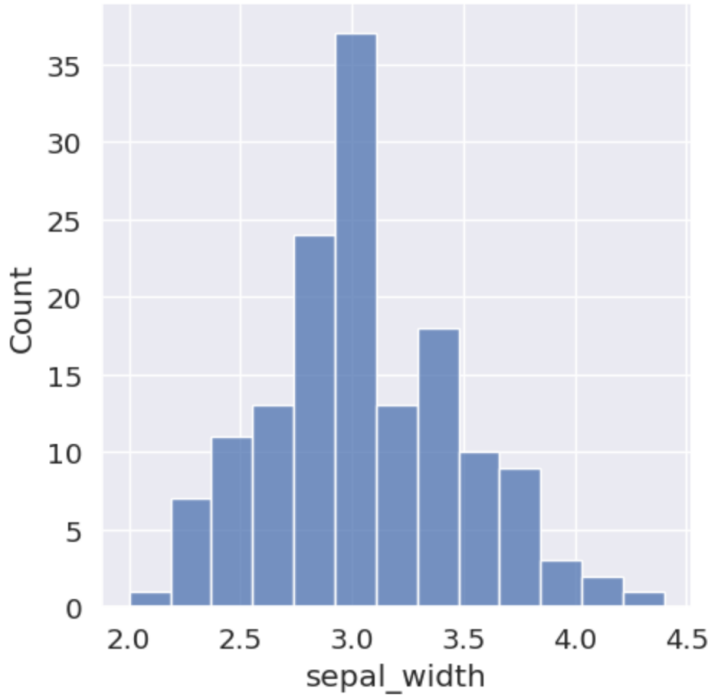
탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 자주 보는 그래프 4가지

[산점도]



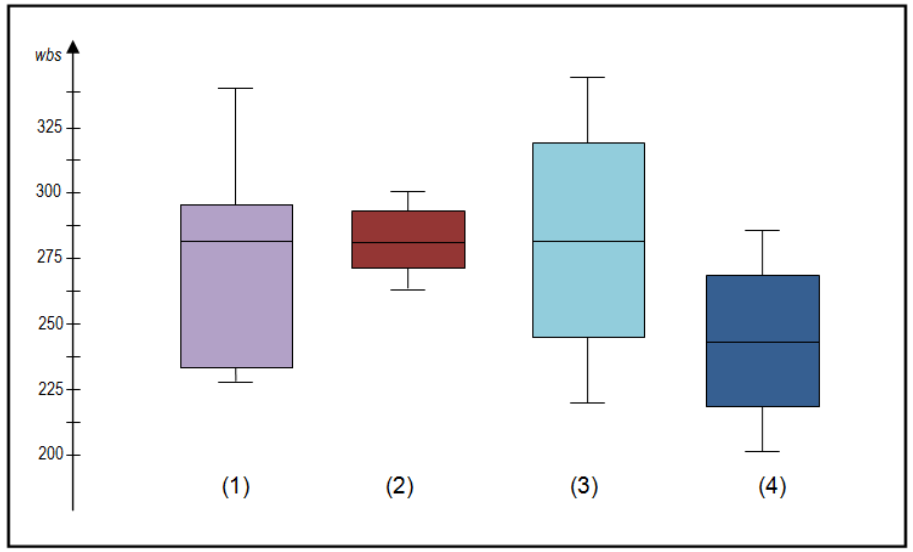
[히스토그램]



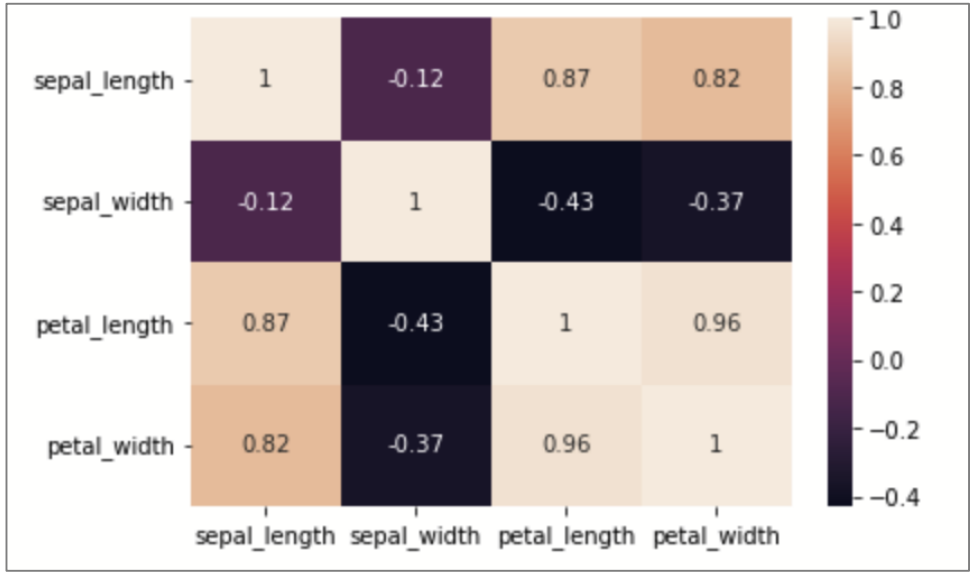
탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 자주 보는 그래프 4가지

[박스플롯]



[상관관계 히트맵]



탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 자주 보는 그래프 4가지

그래프	무엇을 보는가	ML에서의 용도
히스토그램 (Histogram)	한 변수의 값이 어떻게 분포하는지	치우침, 값의 범위, 이상한 분포 확인
박스플롯 (Box Plot)	중앙값, 데이터의 퍼짐, 이상치	이상치 확인, 그룹 간 분포 비교
산점도 (Scatter Plot)	두 수치형 변수의 관계	변수 간 관계, 선형성, 군집 형태 확인
상관관계 히트맵 (Correlation Heatmap)	여러 수치형 변수 간 상관관계	관련성이 큰 변수 파악, 입력 변수 선택 참고

상관관계(Correlation)란?

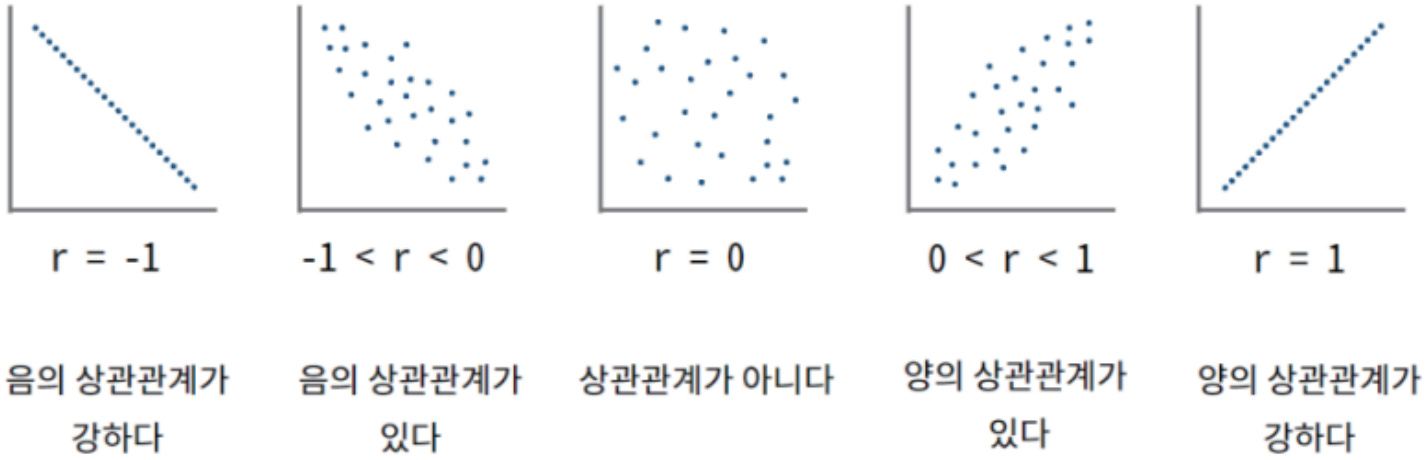
- 개념
  - 두 변수(컬럼) 간에 어떤 선형적 관계가 있는지,
    그리고 그 관계의 강도와 방향이 어떤 지를 나타내는 통계량

- 상관 관계 유형
  - 양의 상관관계 (Positive Correlation) : 한 변수가 증가할 때 다른 변수도 증가하는 경향.
    - 예) 공부 시간과 성적
  - 음의 상관관계 (Negative Correlation) : 한 변수가 증가할 때 다른 변수는 감소하는 경향.
    - 예) 기온과 난방비
  - 상관관계 없음 (No Correlation) : 두 변수 사이에 선형적인 관계가 없는 경우.

상관 계수 (Correlation Coefficient)

- 개념
  - 두 변수(컬럼) 사이의 선형적 관계의 방향과 강도를 -1부터 1 사이의 수치로 나타낸 통계 지표
  - 상관 계수가 높다고 해서 두 변수 사이 인과 관계가 반드시 있는 것은 아니다.

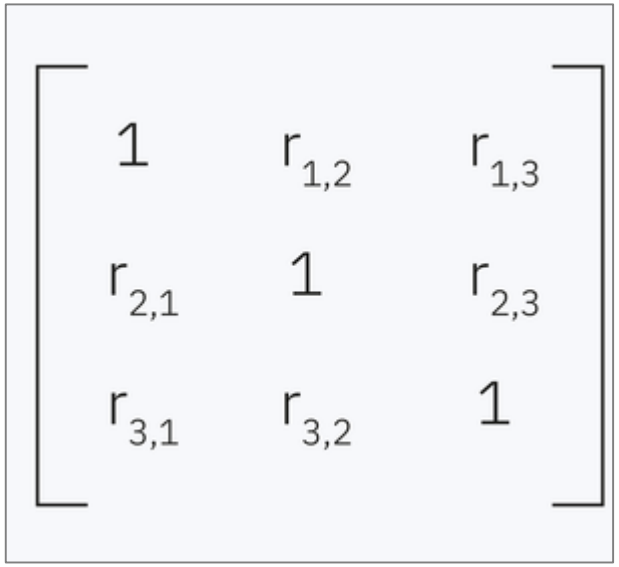
- **상관 계수 값의 범위** : 상관계수는 보통 -1부터 1까지의 값을 가짐



상관 계수 계산하기 : corr() 함수

- pandas.DataFrame.corr()
  - DataFrame 내의 **수치형 칼럼들 간의 상관 관계를 계산하여 → 상관 행렬을 반환**

- 상관 행렬(Correlation Matrix)
  - 각 컬럼 쌍에 대한 상관계수를 행렬로 표현한 것
  - 상관 행렬의 대각선 값은 항상 1
    - 자기 자신과의 상관관계는 항상 1이기 때문



[상관 행렬]

상관 계수 계산하기 : corr() 함수

- 예제

```
import pandas as pd
import numpy as np

# 예시 데이터프레임 생성
data = {'A': [1, 2, 3, 4, 5],
        'B': [2, 4, 5, 4, 5],
        'C': [5, 4, 3, 2, 1],
        'D': [10, 20, 30, 40, 50]}
df = pd.DataFrame(data)

# 상관 행렬 계산
correlation_matrix = df.corr()
print(correlation_matrix)
```

[임의로 생성한 데이터]

A	B	C	D
1	2	5	10
2	4	4	20
3	5	3	30
4	4	2	40
5	5	1	50

[상관 행렬]

```
          A         B         C    D
A  1.000000  0.806226 -1.000000  1.0
B  0.806226  1.000000 -0.806226  0.8
C -1.000000 -0.806226  1.000000 -1.0
D  1.000000  0.806226 -1.000000  1.0
```

왜 상관계수를 구할까?

1. 데이터 탐색 및 시각화

- **변수(컬럼) 간의 관계**(어떤 변수들이 서로 강한 관계를 가지는지)를 빠르게 파악
- 상관 행렬을 히트맵 등으로 시각화하면 **데이터의 패턴을 쉽게 이해**할 수 있음

2. 다중공선성(Multicollinearity) 진단

- 다중공선성 – 회귀 모델에서 **변수(컬럼)들 간에 강한 상관관계가 있을 때 발생하는 문제**를 이르는 말
- 강한 상관관계를 가진 변수들은 모델 성능에 부정적인 영향을 줄 수 있어
  제거하거나 결합하는 등의 작업이 필요

3. 특성 선택(Feature Selection)

- 타겟 변수와 강한 상관 관계를 가지는 특성을 선택하거나,
  상관관계가 낮은 특성을 제거하여 모델의 복잡성을 줄이고 성능을 향상시킬 수 있음

→ 데이터를 파악하여 모델 성능을 향상시키는 방향을 찾기 위해

상관 계수와 다중공선성

- 개념
  - 두 개 이상의 독립 변수들이 서로 강한 선형 관계를 가지고 있는 상태
  - 하나의 독립 변수가 다른 독립 변수들의 선형 조합으로 거의 완벽하게 설명될 수 있는 경우

- 상관 행렬을 통해 다중공선성을 진단할 수 있다.
  - 상관 행렬에서 **타겟 변수(종속 변수)를 제외한 독립 변수들끼리**의 상관 계수 절댓값이 0.8 이상, 또는 0.9 이상으로 매우 높게 나타나는 경우
  - 값이 1에 가깝거나 -1에 가까울수록 두 독립 변수 간에 강한 선형 관계가 있음을 의미

상관 계수와 다중공선성

- 다중공선성이 있는 데이터의 상관 행렬 예시
  - 다중공선성을 진단할 때는 상관 행렬의 대각선 값을 제외하고,
  - 독립 변수들로 구성된 부분 행렬 내에서 높은 상관 계수를 찾아야 합니다.
  - 예시: 다음 상관 행렬에서 다중공선성이 있는 독립변수를 찾아보세요.

	X1	X2	X3	Y
X1	1.00	0.92	0.15	0.70
X2	0.92	1.00	0.20	0.65
X3	0.15	0.20	1.00	0.05
Y	0.70	0.65	0.05	1.00

→ 독립 변수(X) 간에 상관 계수가 높으면 문제가 되는 것이 다중 공선성입니다.

→ 독립 변수(X)와 종속 변수(Y) 간의 상관 계수는 다중공선성과 상관이 없습니다.

→ 오히려 독립 변수(X)와 종속 변수(Y) 간에는 상관 계수가 높을 수록 좋습니다.

왜 다중공선성이 문제인가?

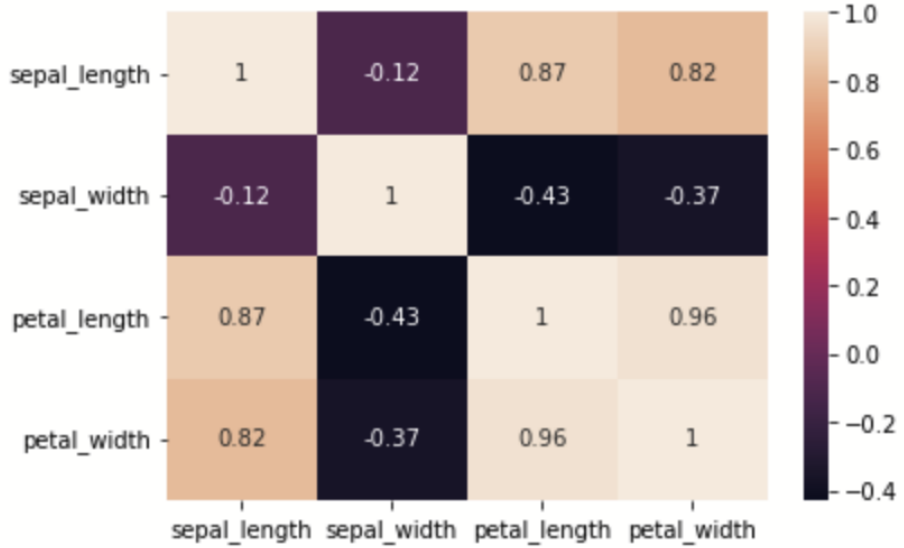
- 가중치(weight / coefficient) 추정의 불안정성
  - : 서로 강하게 관련된 독립 변수들이 많으면, 모델이 각 변수의 개별적인 영향을 정확하게 분리하기 어려워집니다.
  - : 이로 인해 계수 값들이 불안정해지고, 데이터가 조금만 바뀌어도 계수 값이 크게 변동할 수 있습니다.

- 해석의 어려움
  - : 특정 독립 변수가 종속 변수에 미치는 순수한 영향을 해석하기 어려워집니다.
  - : 예) X1과 X2가 강하게 상관되어 있다면,
    X1이 Y에 미치는 영향이 X2 때문에 과대평가되거나 과소평가될 수 있습니다.

- 통계적 유의성 감소
  - : 개별 독립 변수의 p-value이 높아져 통계적으로 유의미하지 않게 나올 수 있습니다.
  - : p-value:
    - 통계적 가설 검정을 통해 특정 관계가 우연에 의한 것인지 아니면 통계적으로 유의미한 것인지를 판단하는 확률 값
    - p-값이 높을수록 (일반적으로 0.05 이상) 해당 독립 변수가 종속 변수에 미치는 영향이 통계적으로 유의미하지 않다고 판단

상관 행렬 시각화 : Heatmap() 함수

- Heatmap() 함수
  - Seaborn 라이브러리에 있는 시각화 함수
  - **데이터를 색상의 강도로 표현**하여 시각화
    - 특히 2차원 매트릭스(행렬) 형태의 데이터를 시각화 할 때 유용
    - 각 셀의 값 크기를 특정 색상 팔레트의 색상 강도로 매핑하여 데이터 패턴 파악에 유용
  - **상관행렬을 히트맵으로 시각화**하면,
    변수 간 관계를 직관적으로 파악하고 다중공선성을 시각적으로 진단하기에 유용



실습 1.
탐색적 데이터 분석

실습 개요

실습 데이터 : 지난 주에 사용한 접착제 공정 데이터의 전처리하기 전 상태

실습 내용

단계 1. 탐색적 데이터 분석을 통해 데이터의 특성을 분석하고 이해하기

단계 2. 데이터에 대한 이해를 바탕으로 어떻게 전처리 할 것인지 결정하기

단계 3. 전처리 이후 로지스틱 회귀 모델로 학습 시키기

실습 목표 : 전처리 전 데이터 탐색의 필요성 이해

문제 정의 : 현장 언어

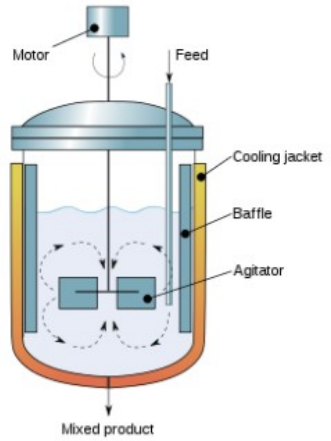
- **화학물 제조공장 현장 문제 파악**
  - 울산에 있는 화학물 제조기업 K사는 접착제를 완제품으로 생산함
  - 특정 공정에서 유해물질이 발생하는 공정이 존재
  - 해당 공정에 문제 발생 시, **양품과 불량을 자동으로 알려주는 머신러닝 모델**이 필요





데이터 수집

- 데이터 수집 전략
  - 대상 공정의 설비인 교반기에 IoT 센서를 부착
  - **온도, 점도를 수집하고 양품일 때와 불량일 때를 구분**



화학물 제조를 위한 교반기



온도, 점도 센서



70개의
제조 생산물(본드)

온도 점도

[[24.41, 1.63
24.46, 1.65
24.52, 1.69
. .
. .
. .
23.73, 0.89]]

<제조데이터 2차원 리스트>

표준 데이터셋 스크립트 : 데이터 수집의 산출물

- 데이터 명세 : **표준 데이터셋 스크립트**

헤더		바디			
No	변수명	척도	유형 타입	변수 설명	속성/범주값
1	Lot	범주형	int64	일정 단위로 생산된 화학물의 고유번호	정수 11자리
2	Temp(°C)	연속형	float64	교반기 내 화학물 용액의 온도	범위 : 20~60°C, 소수점 2자리
3	Viscosity(cP)	연속형	float64	교반기 내 화학물 용액의 점도	범위 : 0~60cP, 소수점 2자리
4	Failure	범주형	int64	화학물 품질 정상/불량 여부	0 : 정상, 1 : 불량

문제 정의 : 공학 언어

헤더		바디			
No	변수명	척도	유형 타입	변수 설명	속성/범주값
1	Lot	범주형	int64	일정 단위로 생산된 화학물의 고유번호	정수 11자리
2	Temp(°C)	연속형	float64	교반기 내 화학물 용액의 온도	범위 : 20~60°C, 소수점 2자리
3	Viscosity(cP)	연속형	float64	교반기 내 화학물 용액의 점도	범위 : 0~60cP, 소수점 2자리
4	Failure	범주형	int64	화학물 품질 정상/불량 여부	0 : 정상, 1 : 불량

- **특성 X** : 온도, 점도
- **타겟 y** : 양품/불량 유무

- **구분:** 지도학습 〉〉 분류

- **적용 가능 알고리즘**
  - K-최근접 이웃
  - 로지스틱 회귀
  - 결정트리
  - 서포트 벡터 머신

기본 분석 : sample, info, describe

```
df.sample(10)
```

	Lot	Temp	Viscosity	Failure
202	2110918638	24.97	3.31	0
334	2109662615	23.94	2.84	0
808	2110886068	24.80	1.11	0
455	2110941645	NaN	2.39	0
686	2110412695	NaN	1.57	0
693	2110253033	23.91	1.54	0
534	2110533550	24.03	2.10	정상
988	2110042367	24.03	NaN	0
64	2109152514	26.81	52.29	1
754	2110753363	24.80	1.34	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Lot        1000 non-null   int64
 1   Temp       870 non-null    float64
 2   Viscosity  911 non-null    float64
 3   Failure    1000 non-null   object
dtypes: float64(2), int64(1), object(1)
memory usage: 31.4+ KB
```

```
df.describe()
```

	Lot	Temp	Viscosity
count	1.000000e+03	870.000000	911.000000
mean	2.110176e+09	31.471437	5.534566
std	6.774789e+05	33.048402	11.889175
min	2.109022e+09	2.450000	0.740000
25%	2.109585e+09	24.352500	1.570000
50%	2.110190e+09	25.430000	2.390000
75%	2.110782e+09	26.440000	3.250000
max	2.111290e+09	435.350000	101.800000

분석 전에 범주형 데이터를 숫자형으로 정리해야 함

- 이 데이터는 Failure가 정리되어 있지 않음

```
# 라벨(Failure) 데이터의 현재 상태 확인
df.Failure.value_counts()
```

Failure	count
0	704
1	120
불량	91
정상	85

dtype: int64

```
# Failure 값 맵핑
df.Failure = df.Failure.map({'정상': 0, '0': 0,
                             '불량': 1, '1': 1})

# 결과 확인
df.info()
df.Failure.value_counts()
```

→ 데이터 탐색하기 전에 Failure 값 정리해야 함

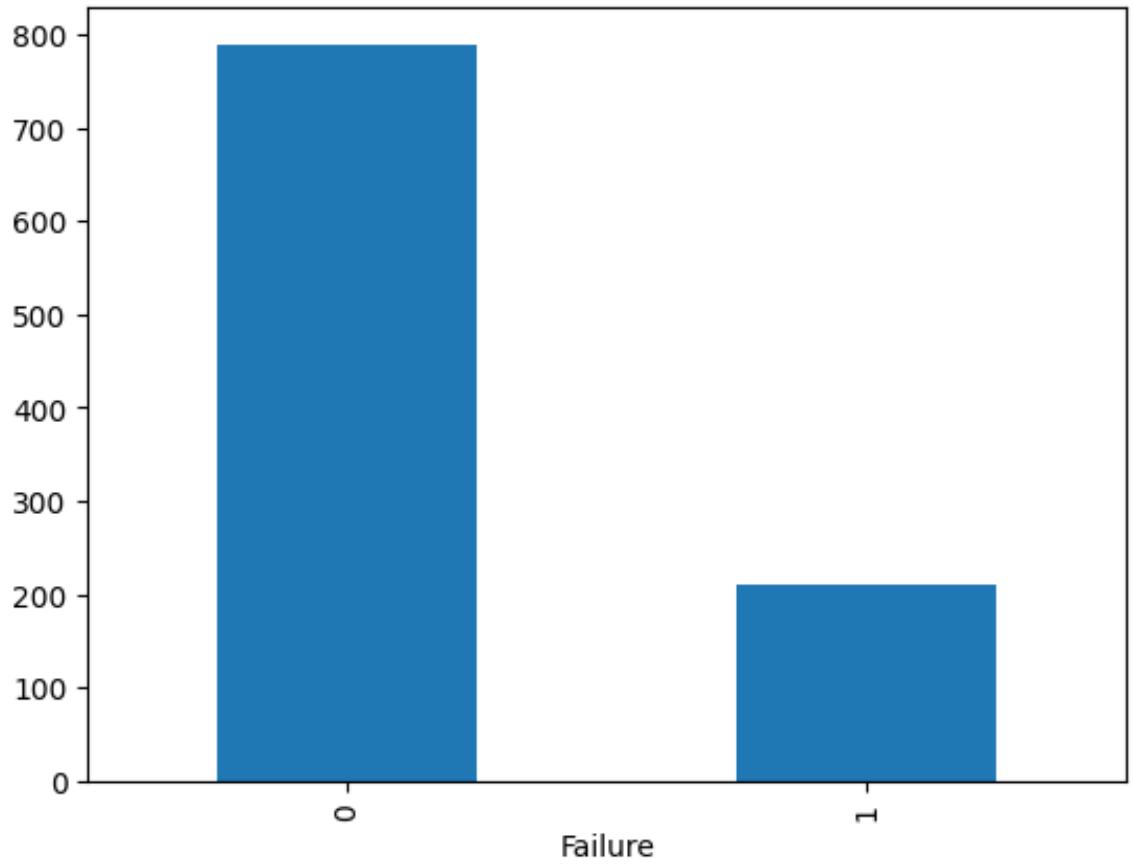
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Lot        1000 non-null   int64
 1   Temp       870 non-null    float64
 2   Viscosity  911 non-null    float64
 3   Failure    1000 non-null   int64
dtypes: float64(2), int64(2)
memory usage: 31.4 KB
```

y 값 데이터 분포

```
import matplotlib.pyplot as plt
import seaborn as sns
```

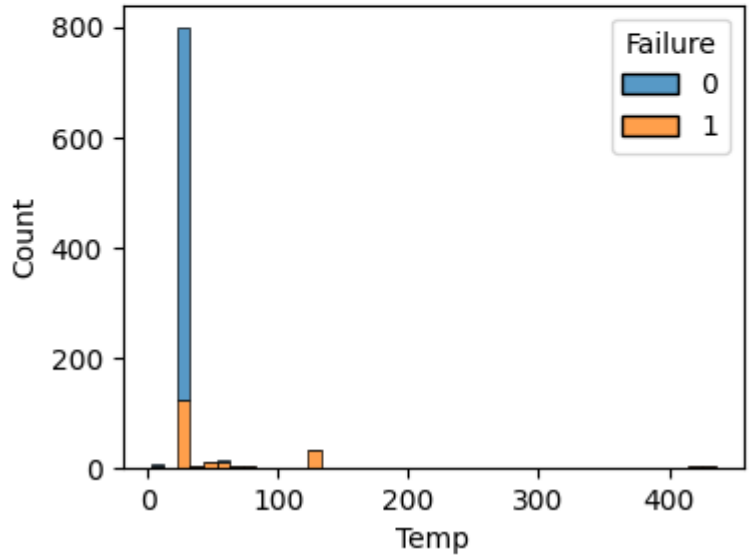
```
# 라벨(y) 분포 확인
df['Failure'].value_counts().plot(kind='bar')

plt.show()
```

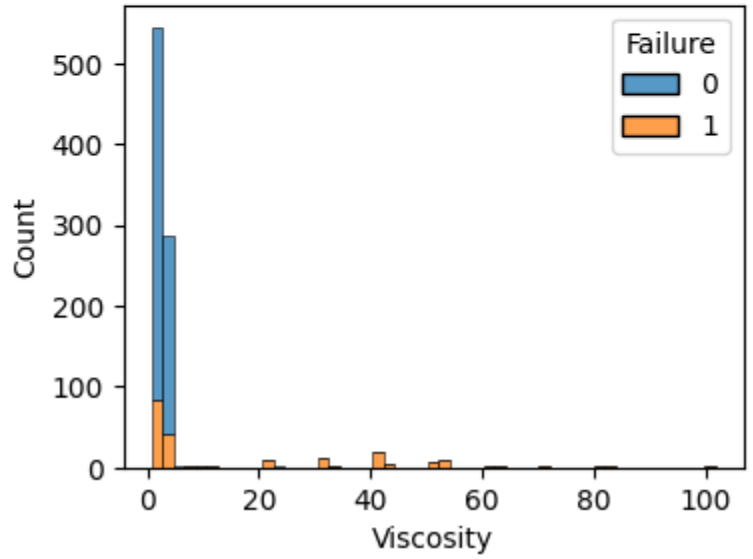


특성(X) 분포 확인

```
# Temp 분포 확인
plt.figure(figsize=(4,3))
sns.histplot(data=df,
             x='Temp',
             hue='Failure',
             multiple='stack',
             binwidth=10)
plt.show()
```

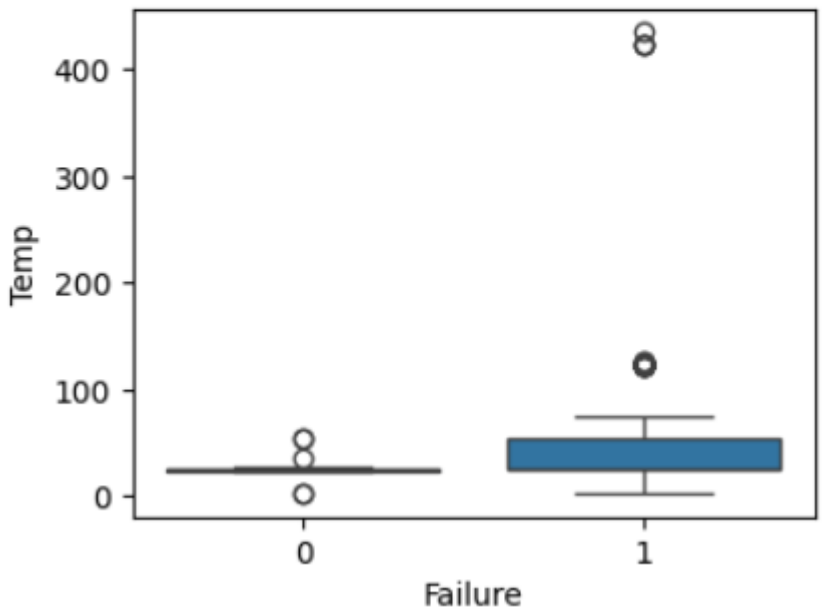



```
# Viscosity 분포 확인
plt.figure(figsize=(4,3))
sns.histplot(data=df,
             x='Viscosity',
             hue='Failure',
             multiple='stack',
             binwidth=2)
plt.show()
```

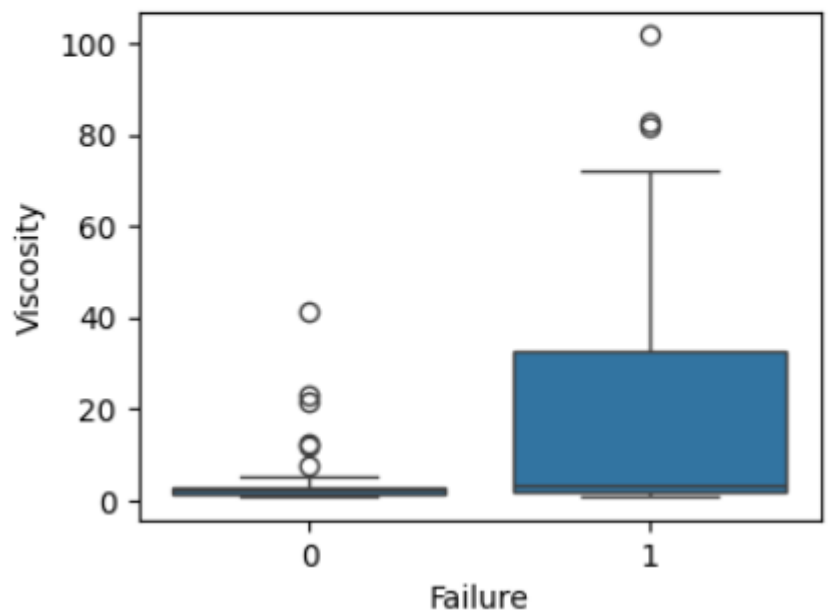


특성(X) 분포 확인

```
# Temp 분포 확인
plt.figure(figsize=(4,3))
sns.boxplot(data=df, x='Failure', y='Temp')
plt.show()
```

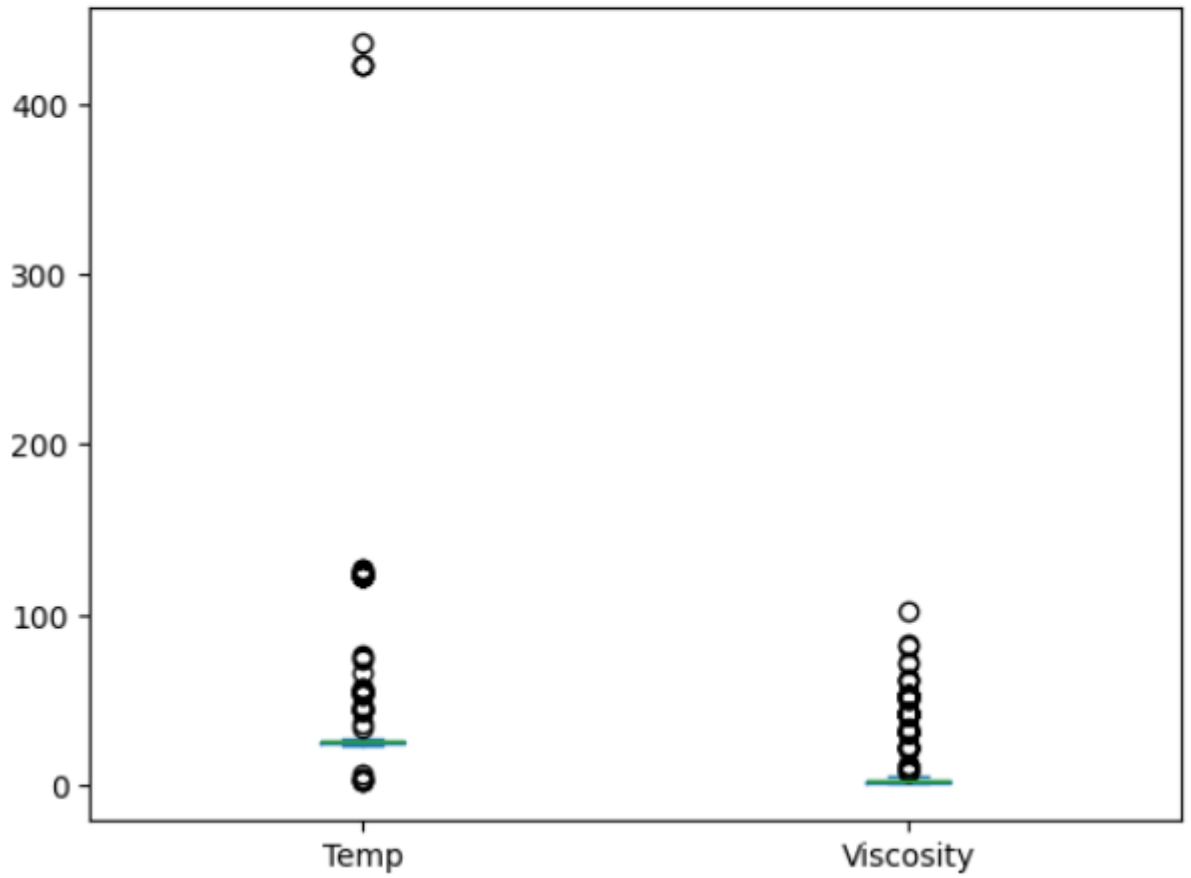



```
# Viscosity 분포 확인
plt.figure(figsize=(4,3))
sns.boxplot(data=df, x='Failure', y='Viscosity')
plt.show()
```



특성(X) 분포 확인

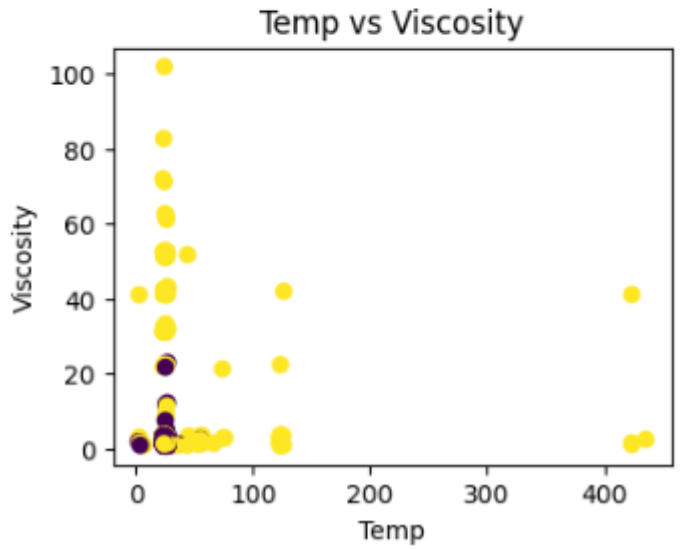
```
# Temp와 Viscosity 분포 비교
df[['Temp', 'Viscosity']].plot(kind='box')
plt.show()
```



Temp와 Viscosity 관계 확인

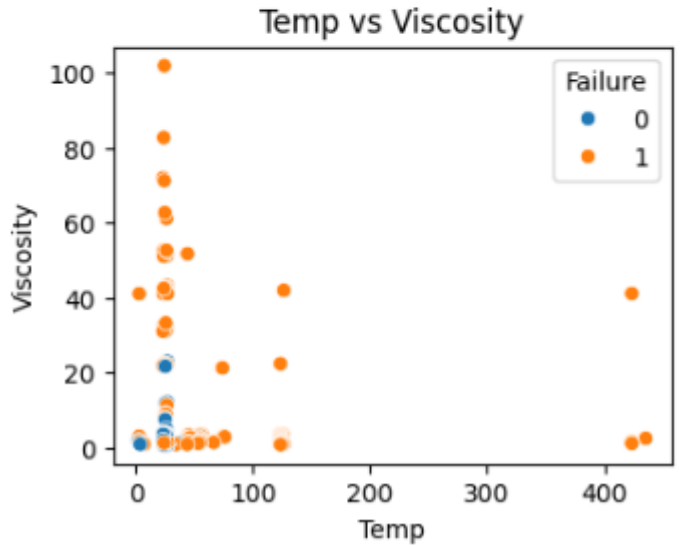
```
#Temp와 Viscosity 관계 확인 - plt 함수
plt.figure(figsize=(4,3))
plt.scatter(df['Temp'],
            df['Viscosity'],
            c=df['Failure'])

plt.title('Temp vs Viscosity')
plt.xlabel('Temp')
plt.ylabel('Viscosity')
plt.show()
```




```
#Temp와 Viscosity 관계 확인 - seaborn 함수
plt.figure(figsize=(4,3))
sns.scatterplot(
    data=df,
    x='Temp',
    y='Viscosity',
    hue='Failure'
)

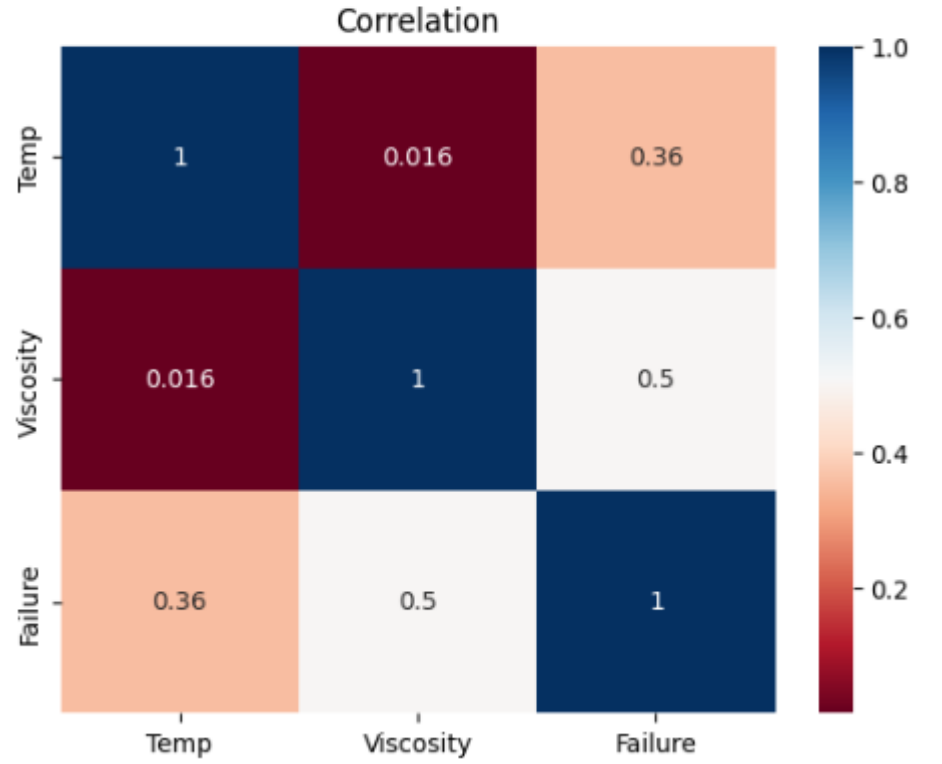
plt.title('Temp vs Viscosity')
plt.show()
```



상관 관계 확인

```
# 상관관계 확인
# Lot는 식별자 성격이므로 제외함
corr = df[['Temp', 'Viscosity', 'Failure']].corr()

sns.heatmap(corr, annot=True,cmap='RdBu')
plt.title('Correlation')
plt.show()
```



결측치 처리 결정하기

- 삭제할 것인가? 대체할 것인가?
- 대체한다면 어느 값으로 대체할 것인가?

```
df.describe()
```

	Lot	Temp	Viscosity
count	1000.0000	870.0000	911.0000
mean	2110175932.5680	31.4714	5.5346
std	677478.9190	33.0484	11.8892
min	2109021818.0000	2.4500	0.7400
25%	2109585209.0000	24.3525	1.5700
50%	2110189868.0000	25.4300	2.3900
75%	2110782208.0000	26.4400	3.2500
max	2111289797.0000	435.3500	101.8000

- 큰 값은 샘플 개수는 적으나 평균에 영향을 많이 미침
- 결측치나 이상치를 산술평균으로 대체하는 것이 적절하지 않음.

→ **중앙값으로 대체하자**

결측치 처리하기

```
# 결측치 확인
df.isnull().sum()
```

	0
Lot	0
Temp	130
Viscosity	89
Failure	0

dtype: int64

```
# 결측치 처리
# 중앙값으로 대체하자.
temp_median = df.Temp.median()
df.Temp = df.Temp.fillna(temp_median)

visco_median = df.Viscosity.median()
df.Viscosity = df.Viscosity.fillna(visco_median)
```

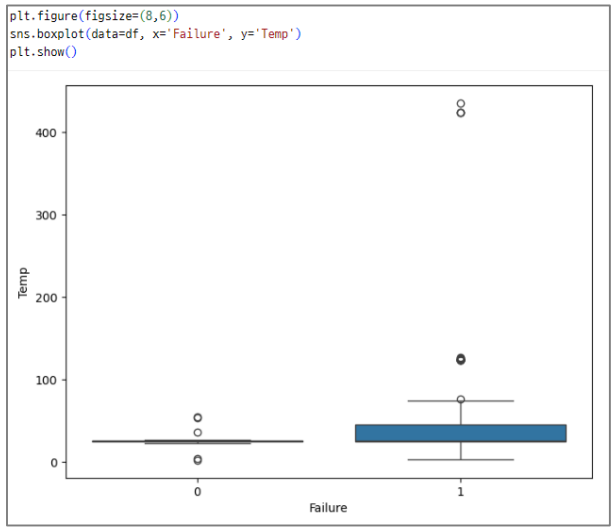
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Lot        1000 non-null   int64
 1   Temp       1000 non-null   float64
 2   Viscosity  1000 non-null   float64
 3   Failure    1000 non-null   int64
dtypes: float64(2), int64(2)
memory usage: 31.4 KB
```

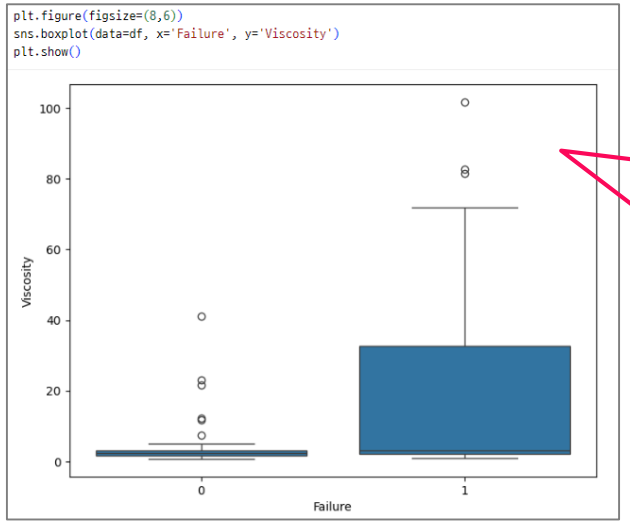
이상치 처리 결정하기

- 삭제할 것인가? 대체할 것인가?
- 대체한다면 어느 값으로 대체할 것인가?

```
plt.figure(figsize=(8,6))
sns.boxplot(data=df, x='Failure', y='Temp')
plt.show()
```




```
plt.figure(figsize=(8,6))
sns.boxplot(data=df, x='Failure', y='Viscosity')
plt.show()
```



- 이상치 자체가 정상/불량을 분류에 영향을 미치는 특성일 수 있음

  → **이상치는 처리하지 않음**

중복 데이터 삭제하기

- 중복값 확인 후 삭제

```
# 중복값이 있는지 확인
df.Lot.duplicated().sum()
```

```
np.int64(131)
```

```
# 중복값 삭제
df = df.drop_duplicates(subset=['Lot'], keep='first')
```

- 삭제 전후 데이터 비교

삭제 전

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Lot          1000 non-null   int64
 1   Temp         1000 non-null   float64
 2   Viscosity    1000 non-null   float64
 3   Failure      1000 non-null   int64
 4   FailureCode  1000 non-null   object
dtypes: float64(2), int64(2), object(1)
memory usage: 39.2+ KB
```

삭제 후

```
<class 'pandas.core.frame.DataFrame'>
Index: 869 entries, 0 to 999
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Lot          869 non-null    int64
 1   Temp         869 non-null    float64
 2   Viscosity    869 non-null    float64
 3   Failure      869 non-null    int64
 4   FailureCode  869 non-null    object
dtypes: float64(2), int64(2), object(1)
memory usage: 40.7+ KB
```

데이터 표준화

- 데이터 표준화는 X에만 적용하므로, **입력 데이터(X)와 정답 라벨(y)를 분리한 후에** 작업한다.

```
# 입력 데이터(X)와 정답 데이터(y) 분리
X = df[['Temp', 'Viscosity']]
y = df['Failure']

# 학습용 / 테스트용 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# 데이터 표준화
from sklearn.preprocessing import StandardScaler

# 표준화 객체 생성
scaler = StandardScaler()

# 학습 데이터로 기준을 학습하고 표준화
X_train_scaled = scaler.fit_transform(X_train)

# 테스트 데이터는 학습 데이터의 기준으로 표준화
X_test_scaled = scaler.transform(X_test)
```

https://chatgpt.com/s/w_6aa2e757bdc48191869a87ee3c53444d

전처리가 완료된 데이터셋으로 로지스틱 회귀 모델 학습

문제 정의

- **특성 X :** 온도, 점도
- **타겟 y :** 양품/불량 유무
- **구분:** 지도학습 〉〉 분류

실습 내용

1. 로지스틱 회귀 모델을 사용하여 이진 분류 모델을 학습시키기
2. 학습 후 모델의 학습 결과인 coef_와 intercept_ 값 확인하기
3. 모델의 성능 지표(정확도) 확인하기

[참고] Matplotlib.pyplot 에서 출력 문제 설정

- 한글 깨짐과 마이너스 깨짐 방지 코드

```
# 1. 아래 코드 실행 후, 코랩 재시작
!sudo apt-get install -y fonts-nanum
!sudo fc-cache -fv
!rm ~/.cache/matplotlib -rf
```

```
# 2. 재시작 후 아래 코드를 실행
import matplotlib.pyplot as plt
plt.rc('font', family='NanumBarunGothic')
plt.rcParams['axes.unicode_minus'] = False  # 마이너스 깨짐 방지
```

2. 모델 학습 결과 해석하기

무엇을, 어떻게 해석해야 하는가?

- 모델의 학습 결과를 해석할 때 고려해야 하는 것들
  - **데이터의 특성 :** 불균형 데이터의 이진 분류 모델
  - **모델의 성능지표 : 혼동 행렬**
  - **과대적합/과소적합 여부**
  - **모델이 학습한 가중치 값**

- 모델의 학습 결과를 해석하는 과정에서 나와야 하는 것은?
  - **현재 모델은 목표한 성능을 만족하고 있는가?**
  - **현재 모델의 성능에 영향을 미친 요소는 무엇인가?**
  - **모델 성능 개선하기 위해 무엇을 시도해볼 것인가?**

불균형 데이터의 성능 평가

- 정확도(accuracy) 만으로는 부족하다!
  - 데이터셋이 불균형할 때 (예: 정상 메일 99%, 스팸 메일 1%),
  - 모델이 모든 메일을 '정상'으로 예측해도 정확도는 99%
  - 하지만 실제 스팸 메일은 하나도 걸러내지 못하는 문제가 발생

성능 평가: 혼동 행렬(Confusion Matrix)

- 모델의 예측 결과를 실제 값과 비교하여 시각적으로 보여주는 표
- 분류 모델의 예측 성능을 평가하는 데 사용

		Predicted Class: P	Predicted Class: N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

- TP/진양성 : 실제 양성, 예측 양성
- TN/진음성 : 실제 음성, 예측 음성
- FP/위양성 : 실제 음성, 예측 양성 (거짓 양성)
- FN/위음성 : 실제 양성, 예측 음성 (거짓 음성)

성능 평가: 혼동 행렬

		Predicted Class P	Predicted Class N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

- 정확도 (Accuracy)
  - 전체 중 정답을 맞춘 비율

$$Accuracy = \frac{TP + TN}{\text{전체}}$$

- 불균형 데이터에서는 정확도(accuracy) 만으로는 부족하다!
  - 데이터셋이 불균형할 때 (예: 정상 메일 99%, 스팸 메일 1%),
  - 모델이 모든 메일을 '정상'으로 예측해도 정확도는 99%
  - 하지만 실제 스팸 메일은 하나도 걸러내지 못하는 문제가 발생

성능 평가: 혼동 행렬

- 유방암 진단과 같은 문제에서 위양성과 위음성
  - FP (위양성): 실제로는 암이 아닌데 암이라고 진단하는 경우
    : 환자에게 불필요한 공포와 추가적인 검사 비용을 발생시킴

  - FN (위음성): 실제로는 암인데 암이 아니라고 진단하는 경우
    : 환자의 치료 시기를 놓치게 하여 생명을 위협할 수 있는 가장 치명적인 오류

→ 정확도가 99%라도 모든 양성 사례를 놓치고 있다면 (FN이 높다면) 쓸모가 없을 수 있음

→ 문제의 특성에 따라 다른 평가 지표를 고려해야함.

→ 혼동행렬을 통해 정확도만으로는 알 수 없는 정보를 제공받을 수 있음!

성능 평가: 혼동 행렬

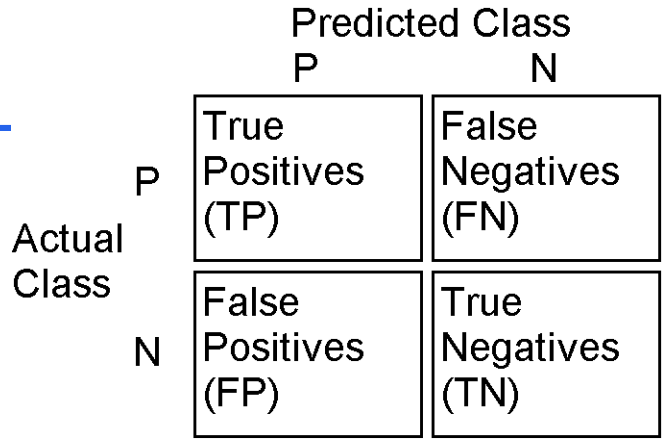
		Predicted Class P	Predicted Class N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

- 정밀도(Precision)
  - 맞다고 한 것 중 진짜 맞은 비율
  - '양성'이라고 예측한 것들 중에서 실제로 '양성'인 비율

$$\text{정밀도} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

  - FP(위양성)를 줄이는 것이 중요한 상황에서 평가 지표로 사용됨
    - 스팸 메일: 스팸이 아닌 중요한 메일을 스팸으로 분류(FP)하면 사용자에게 불편을 줌
    - 자율 주행: 보행자가 아닌 물체를 보행자로 잘못 인식(FP)하여 급정거하는 상황 등

성능 평가: 혼동 행렬



- 재현율 (Recall)
  - 실제 맞는 것 중 얼마나 찾아냈는가
  - 실제 '양성'인 것들 중에서 모델이 '양성'이라고 예측한 비율

$$\text{재현율} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

  - FN(위음성)을 줄이는 것이 중요한 상황에서 성능 지표로 사용
    - 예) 유방암 진단, 금융 사기 탐지

성능 평가: 혼동 행렬

- F1 Score
  - 정밀도와 재현율의 조화 평균
  - 정밀도와 재현율이 한쪽으로 치우치지 않고 모두 높은 값을 가질 때 높은 점수를 얻음

$$\text{F1-Score} = 2 \times \frac{\text{정밀도} \times \text{재현율}}{\text{정밀도} + \text{재현율}}$$

  - 데이터셋에 클래스 불균형이 심할 때, 또는 정밀도와 재현율 모두 중요한 상황에서 모델의 성능을 하나의 숫자로 표현하고 싶을 때 유용

머신러닝은 “얼마나 많이 맞췄는가” 보다

“무엇을 틀렸는가”가 더 중요하다

실습 2.
모델 학습 결과 해석하기

혼동행렬 확인하기

- 실습 1. 에서 학습한 로지스틱 모델의 혼동행렬 값을 확인해보자.

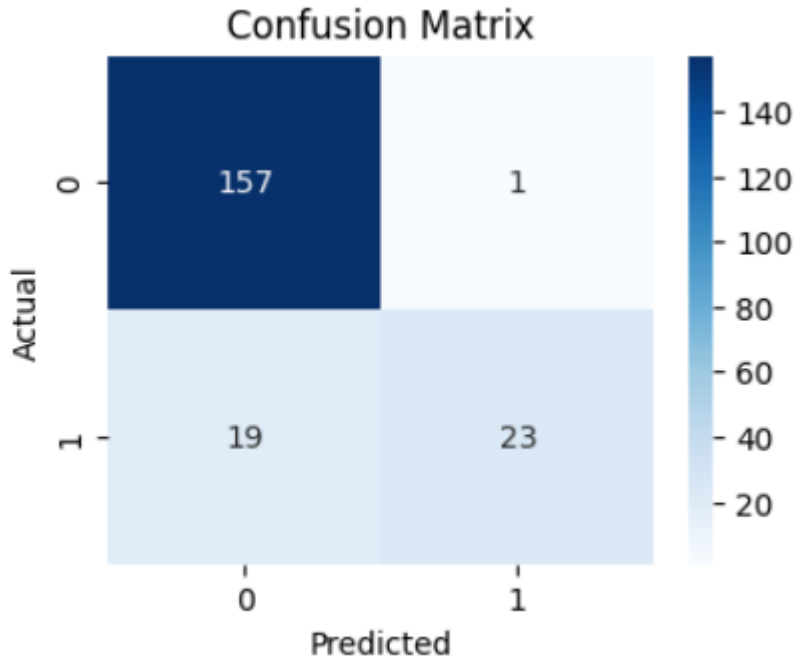
```
# 혼동행렬 확인
cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix')
print(cm)
```

```
Confusion Matrix
[[157   1]
 [ 19  23]]
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# 혼동행렬 시각화
plt.figure(figsize=(4,3))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



성능 지표 확인하기

- 실습 1. 에서 학습한 로지스틱 모델의 성능 지표를 확인해보자.

```python
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

# 성능 지표 확인 - 개별 함수 사용
print('Accuracy :', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall   :', recall_score(y_test, y_pred))
print('F1 Score :', f1_score(y_test, y_pred))
```

```
Accuracy : 0.9
Precision: 0.9583333333333334
Recall   : 0.5476190476190477
F1 Score : 0.696969696969697
```

지표	의미	예지보전에서의 질문
Accuracy	전체 예측 중 맞힌 비율	전체적으로 얼마나 잘 맞혔나?
Precision	고장이라고 예측한 것 중 실제 고장 비율	고장 경고를 했을 때 얼마나 믿을 만한가?
Recall	실제 고장 중 찾아낸 비율	실제 고장을 얼마나 놓치지 않았나?
F1-score	Precision과 Recall의 균형	두 성능을 종합하면 어떤가?

성능 지표 확인하기

- 실습 1. 에서 학습한 로지스틱 모델의 성능 지표를 확인해보자.

```
# 성능 지표 확인 - 좀 더 간편한 방법
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.89      0.99      0.94       158
           1       0.96      0.55      0.70        42

    accuracy                           0.90       200
   macro avg       0.93      0.77      0.82       200
weighted avg       0.91      0.90      0.89       200
```

coef_와 intercept_ 값 확인하기

- 어느 특성이 고장(Failure)에 얼만큼 영향을 미치는 지 알 수 있다.

```
# 가중치 확인
print('Weights:', model.coef_)

# 절편 확인
print('Bias:', model.intercept_)
```

```
Weights: [[2.97763375 1.66866203]]
Bias: [-1.16037744]
```

```
# 가중치와 절편 더 보기 편하게 출력하기
for feature, weight in zip(X.columns, model.coef_[0]):
    print(feature, ':', weight)

print('Bias:', model.intercept_[0])
```

```
Temp : 2.9776337510532414
Viscosity : 1.6686620309460258
Bias: -1.1603774402128728
```

실습 3.
: 다항 특성 만들기

다항 특성 만들기 : PolynomialFeatures

- 개념
  - 기존 특성들을 조합하여 다항 특성을 생성해주는 클래스

- 사용 방법
  1. PolynomialFeatures 객체 생성
  2. fit_transform 함수로 특성 변환

원래 특성	새로 생성된 특성
x_1	x_1
x_2	x_2
	x_1^2
	x_2^2
	x_1x_2

다항 특성 만들기 : PolynomialFeatures

- 예제 코드 :

```
# 다항 특성 생성 예시
from sklearn.preprocessing import PolynomialFeatures

# 2개의 특성을 가진 데이터
X = [[2, 3], [4, 5]]

# degree=2인 PolynomialFeatures 객체 생성
poly = PolynomialFeatures(degree=2)

# 특성 변환
X_poly = poly.fit_transform(X)

print(X_poly)
# 출력: [[1., 2., 3., 4., 6., 9.], [1., 4., 5., 16., 20., 25.]]
```

다항 특성 만들기 : PolynomialFeatures

1. PolynomialFeatures 객체 생성 → 특성을 어떻게 생성할 것인지 설정

```
# degree=2인 PolynomialFeatures 객체 생성
poly = PolynomialFeatures(degree=2)
```

→

원래 특성	새로 생성된 특성
x_1	x_1
x_2	x_2
	x_1^2
	x_2^2
	x_1x_2

- degree : 생성할 다항식의 차수
  - 1 : 기존 특성 그대로
  - 2 : 2차항(x^2)과 상호작용 항(x1 * x2)까지 생성
  - 3 : 3차항(x^3)과 더 복잡한 상호작용 항까지 생성

2. fit_transform 함수로 특성 변환 → 실제로 변환 작업을 수행

```
# 특성 변환
X_poly = poly.fit_transform(X)
```

실습 내용

1. PolynomialFeatures로 데이터에 다항 특성 추가하기
2. 다항 특성이 추가된 데이터로 로지스틱 회귀 학습시키기
3. 다항 특성이 모델 성능 개선에 도움이 되는지 확인하기

다항 특성 추가하기

- 작업 순서에 유의해야 한다.
  - train/test 데이터 분할 → 다항 특성 추가 → 표준화 적용

```
# 2차 다항 특성 생성
poly = PolynomialFeatures(
    degree=2,
    include_bias=False
)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```

```
# 어떤 변수가 만들어졌는지 확인하기
print(poly.get_feature_names_out(X.columns))

# 결과는 다음과 같은 형태임
# ['Temp'
#  'Viscosity'
#  'Temp^2'
#  'Temp Viscosity'
#  'Viscosity^2']
```

```
['Temp' 'Viscosity' 'Temp^2' 'Temp Viscosity' 'Viscosity^2']
```

```
# 데이터 표준화
from sklearn.preprocessing import StandardScaler

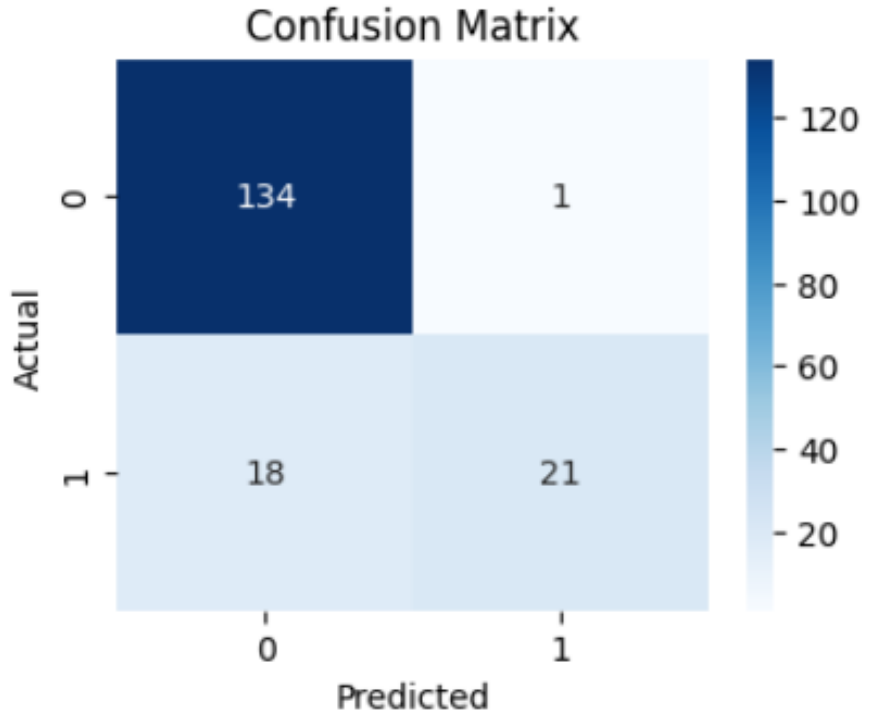
# 표준화 객체 생성
scaler = StandardScaler()

# 학습 데이터로 기준을 학습하고 표준화
X_train_scaled = scaler.fit_transform(X_train_poly)

# 테스트 데이터는 학습 데이터의 기준으로 표준화
X_test_scaled = scaler.transform(X_test_poly)
```

결과 확인 및 분석

- 혼동 행렬과 성능 평가



	precision	recall	f1-score	support
0	0.88	0.99	0.93	135
1	0.95	0.54	0.69	39
accuracy			0.89	174
macro avg	0.92	0.77	0.81	174
weighted avg	0.90	0.89	0.88	174

결과 확인 및 분석

- 다항 특성별 가중치 확인

```
# 다항 특성별 가중치 확인
feature_names = poly.get_feature_names_out(X.columns)

for feature, weight in zip(feature_names, model_poly.coef_[0]):
    print(f'{feature:20s}: {weight:.4f}')

print(f'Bias                 : {model_poly.intercept_[0]:.4f}')
```

```
Temp                 : 2.9943
Viscosity            : 1.3283
Temp^2               : 0.7586
Temp Viscosity       : 0.7349
Viscosity^2          : 0.2620
Temp^3               : 0.1476
Temp^2 Viscosity     : 0.1096
Temp Viscosity^2     : 0.1014
Viscosity^3          : -0.0543
Bias                 : -0.8775
```

→로지스틱 모델은 이상치에 민감한 모델

→성능에 한계가 있을 수 있다.

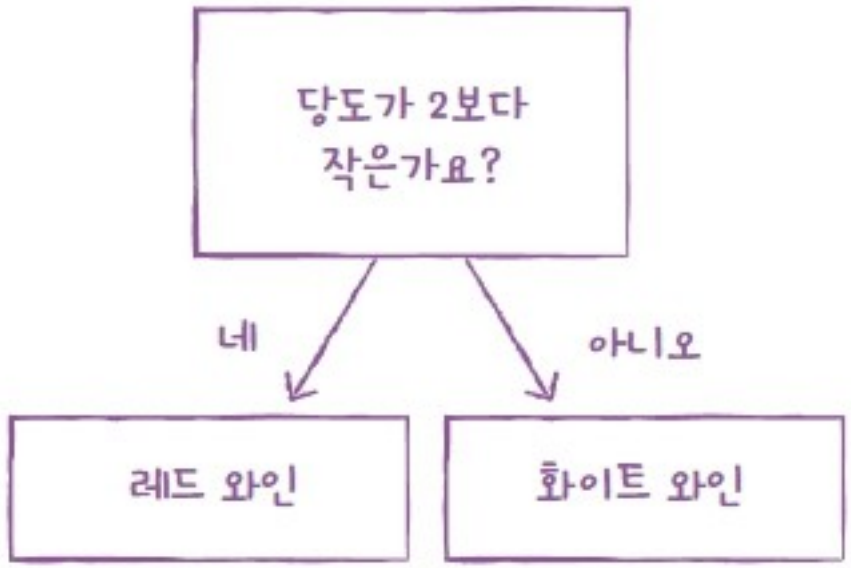
→이상치에 강건한 트리 기반 모델을 써야 할 때다

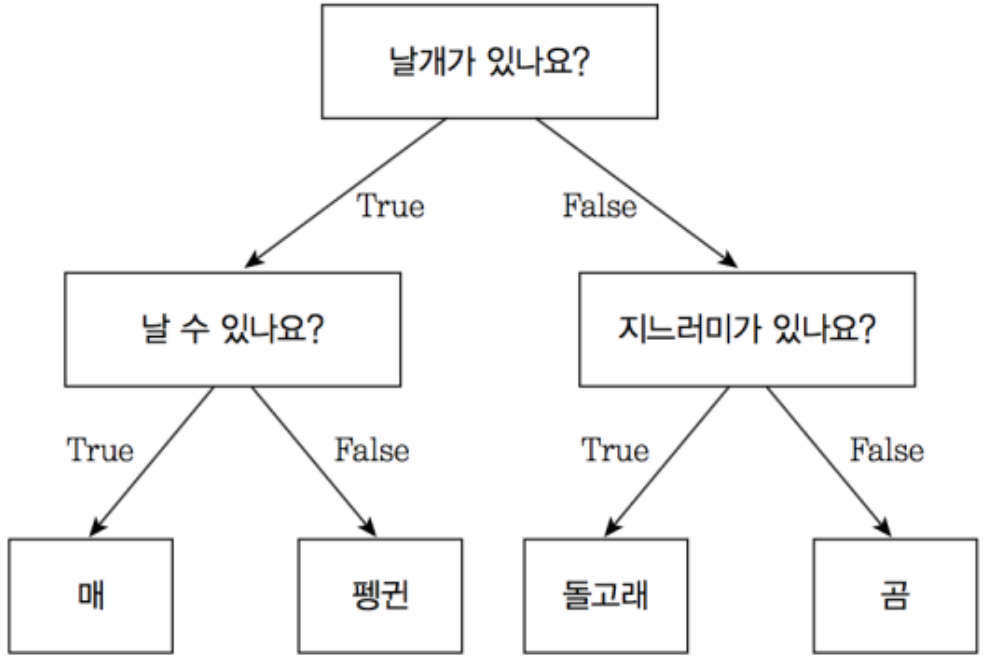
실습 4.

: 결정 트리 모델

결정 트리 (Decision Tree)

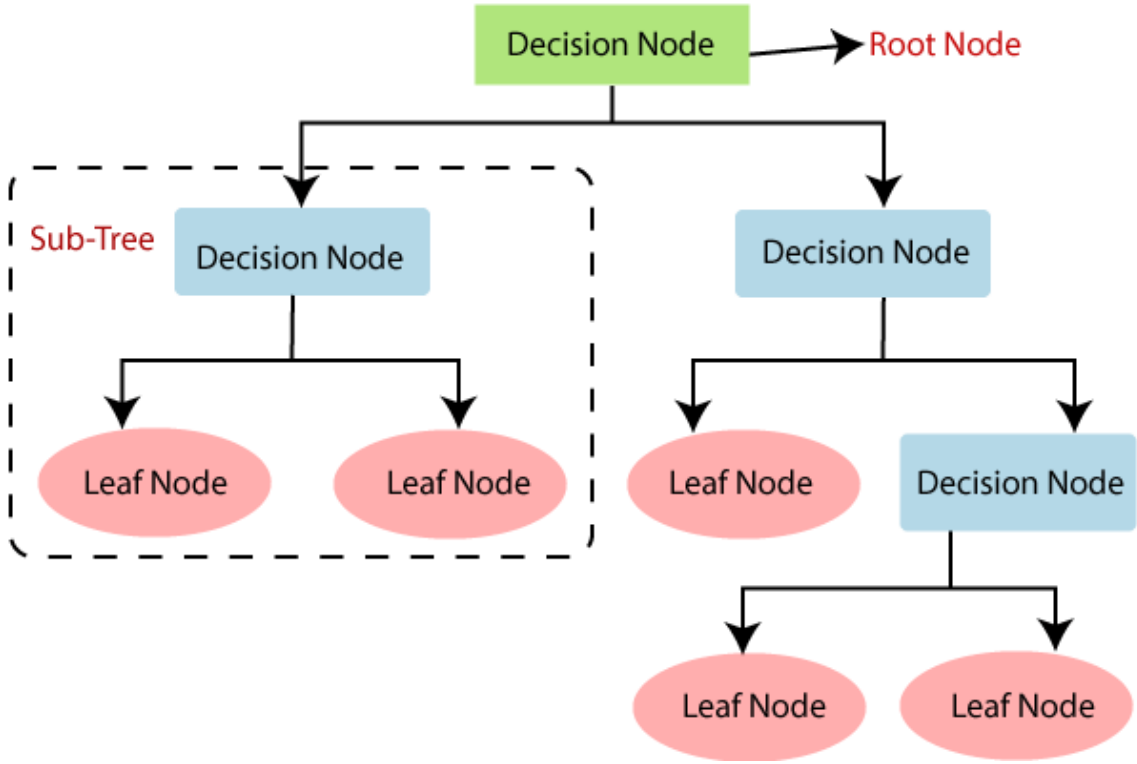
- 개념
  - 여러 가지 규칙을 통해 데이터를 분류하거나 예측하는 데 사용되는 트리 형태의 모델
  - 특정 질문에 "예/아니오"로 답하며 가지를 뻗어 나가듯이 데이터를 분류해 나감





결정 트리 구성요소

- 뿌리 노드(Root Node)
  - : 트리의 가장 위에 있는 노드.
  - : 데이터 전체를 대표하며, 분류를 시작하는 첫 번째 질문이 있는 곳
- 가지(Branch)
  - : 각 질문에 대한 답변에 따라 노드에서 뻗어 나가는 연결선
- 서브 노드(Sub Node)
  - : 뿌리 노드와 잎사귀 노드 사이에 있는 노드.
  - : 또 다른 질문을 통해 데이터를 더 세분화하는 역할
- 잎사귀 노드(Leaf Node 또는 Terminal Node)
  - : 트리의 가장 아래에 있는 노드
  - : 더 이상 질문이 없고 최종적인 분류 결과가 결정되는 곳



https://velog.io/@sset2323/04-02.-%EA%B2%B0%EC%A0%95-%ED%8A%B8%EB%A6%AC

결정 트리 _불순도(impurity)

- 작동 원리
  - 불순도(Impurity)라는 개념을 사용
    - 불순도가 낮다 한 노드 안에 같은 범주의 데이터가 많이 모여 있다
    - 불순도가 높다 한 노드 안에 다양한 범주의 데이터가 섞여 있다
    - 주요 불순도 측정 방법: 지니 불순도(Gini Impurity), 엔트로피(Entropy)

$$\text{지니 불순도} = 1 - (\text{음성 클래스 비율}^2 + \text{양성 클래스 비율}^2)$$

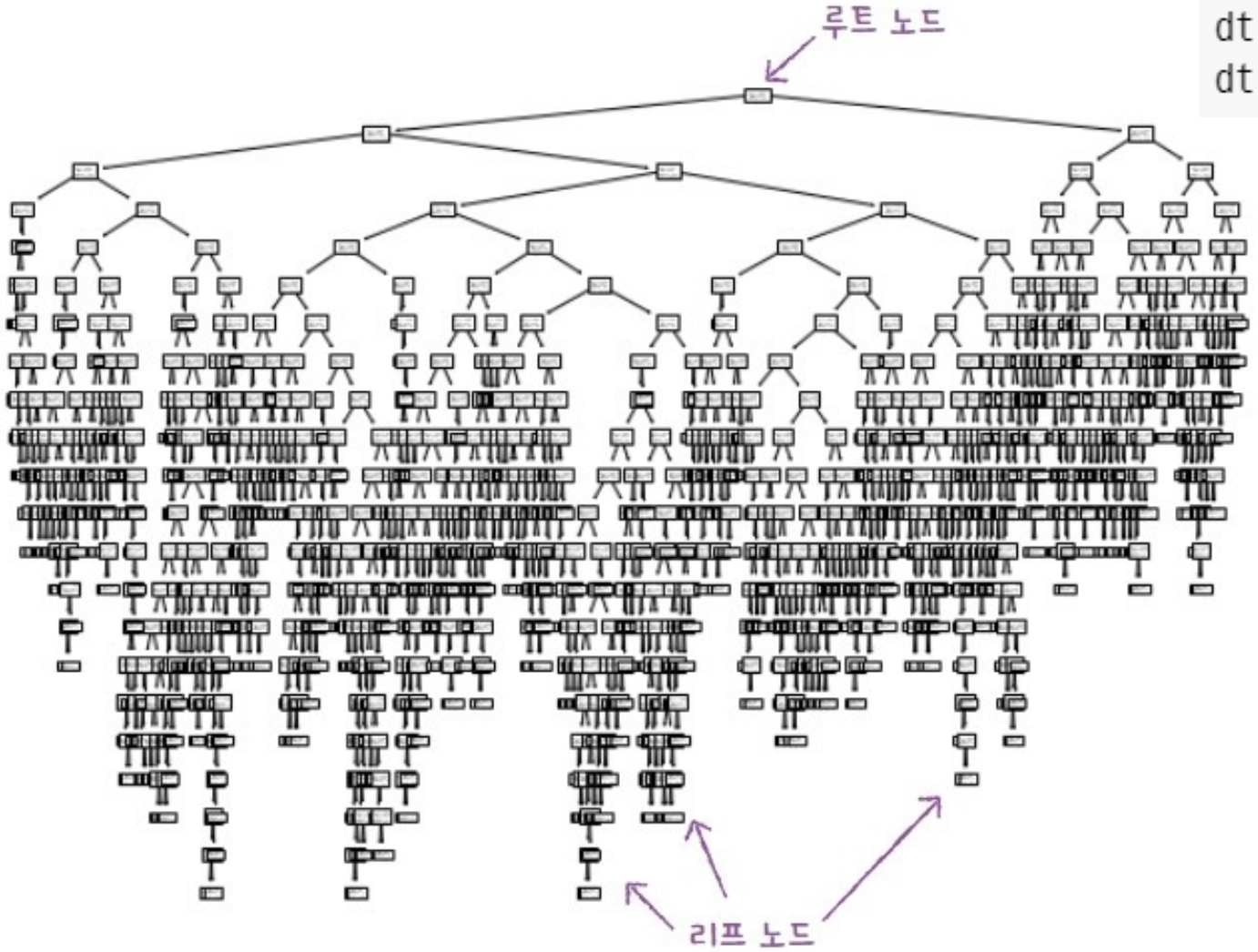
  - 각 노드에서 데이터를 분할할 때 불순도를 가장 많이 감소시키는 질문을 찾아 분할
    - 정보 이득(부모 노드와 자식 노드의 불순도 차이)이 최대가 되도록 데이터를 분할

$$\text{부모의 불순도} - (\text{왼쪽 노드 샘플 수} / \text{부모의 샘플 수}) \times \text{왼쪽 노드 불순도} - (\text{오른쪽 노드 샘플 수} / \text{부모의 샘플 수}) \times \text{오른쪽 노드 불순도} =$$

  - 이 과정을 반복하여 잎사귀 노드에서는 최대한 순수한 데이터 묶음이 되도록 만듦

결정 트리 (Decision Tree)

- DecisionTreeClassifier 클래스




```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)
dt.fit(train_scaled, train_target)
```

결정 트리 구현 : DecisionTreeClassifier

- criterion (불순도 측정 기준)
  - "gini" (기본값) : 지니 불순도(Gini impurity)를 사용하여 분할 품질을 측정합니다.
  - "entropy": 정보 이득(information gain)을 사용하여 분할 품질을 측정합니다.
- splitter (분할 전략) :
  - "best" (기본값) : 각 노드에서 최적의 분할을 찾기 위해 모든 특성을 검토합니다.
  - "random": 각 노드에서 무작위로 일부 특성을 선택하여 최적의 분할을 찾습니다.
- max_depth (최대 깊이) :
  - 트리의 최대 깊이를 제한합니다.
  - None (기본값)이면 잎사귀 노드가 순수해지거나, min_samples_split 미만의 샘플을 포함할 때까지 트리를 성장시킵니다.

Baseline Model : 로지스틱 회귀

```python
# baseline model
# 데이터 표준화
# 표준화 객체 생성
scaler = StandardScaler()

# 학습 데이터로 기준을 학습하고 표준화
X_train_scaled = scaler.fit_transform(X_train)

# 테스트 데이터는 학습 데이터의 기준으로 표준화
X_test_scaled = scaler.transform(X_test)

# 로지스틱 회귀 모델 생성
model_logi = LogisticRegression()

# 모델 학습
model_logi.fit(X_train_scaled, y_train)

# 테스트 데이터 예측
y_pred_logi = model_logi.predict(X_test_scaled)
```

```python
# 혼동행렬
cm_logi = confusion_matrix(y_test, y_pred_logi)

print('Confusion Matrix')
print(cm_logi)

# 분류 성능 지표
print(classification_report(y_test, y_pred_logi))
```

```
Confusion Matrix
[[134   1]
 [ 18  21]]
              precision    recall  f1-score   support

           0       0.88      0.99      0.93       135
           1       0.95      0.54      0.69        39

    accuracy                           0.89       174
   macro avg       0.92      0.77      0.81       174
weighted avg       0.90      0.89      0.88       174
```

결정 트리 모델

```
# 라이브러리 불러오기
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
# 결정 트리 모델 생성
model_tree = DecisionTreeClassifier(
    random_state=42
)

# 모델 학습
model_tree.fit(X_train, y_train)

# 예측
y_pred_tree = model_tree.predict(X_test)
```

결정 트리 모델 지표 확인

```python
# 혼동행렬
cm_tree = confusion_matrix(y_test, y_pred_tree)

print('Confusion Matrix')
print(cm_tree)

# 분류 성능 지표
print(classification_report(y_test, y_pred_tree))
```

```
Confusion Matrix
[[122  13]
 [ 15  24]]
              precision    recall  f1-score   support

           0       0.89      0.90      0.90       135
           1       0.65      0.62      0.63        39

    accuracy                           0.84       174
   macro avg       0.77      0.76      0.76       174
weighted avg       0.84      0.84      0.84       174
```

```python
# 특성 중요도(feature importance) 확인
for feature, importance in zip(
    X.columns,
    model_tree.feature_importances_
):
    print(f'{feature}: {importance:.4f}')
```

```
Temp: 0.4939
Viscosity: 0.5061
```

4. 숫자 제조데이터 기반 AI 개발

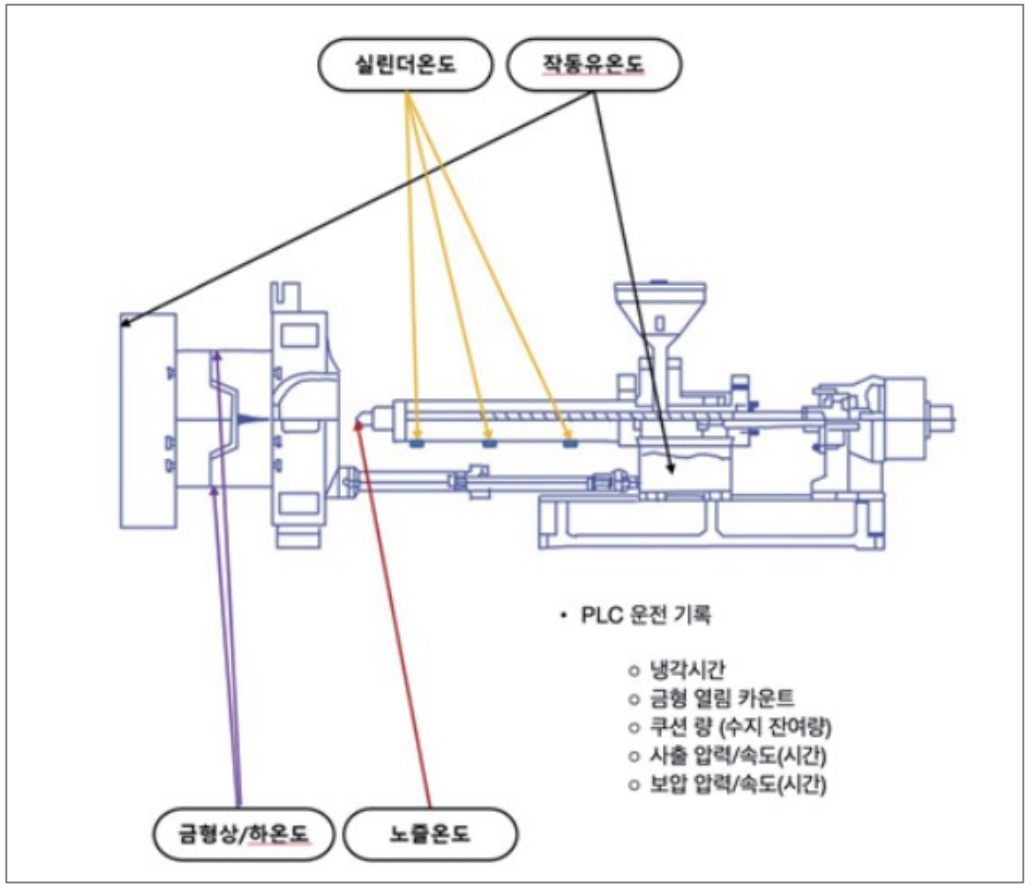
(교재 Chapter 5)

사출성형 제조공장

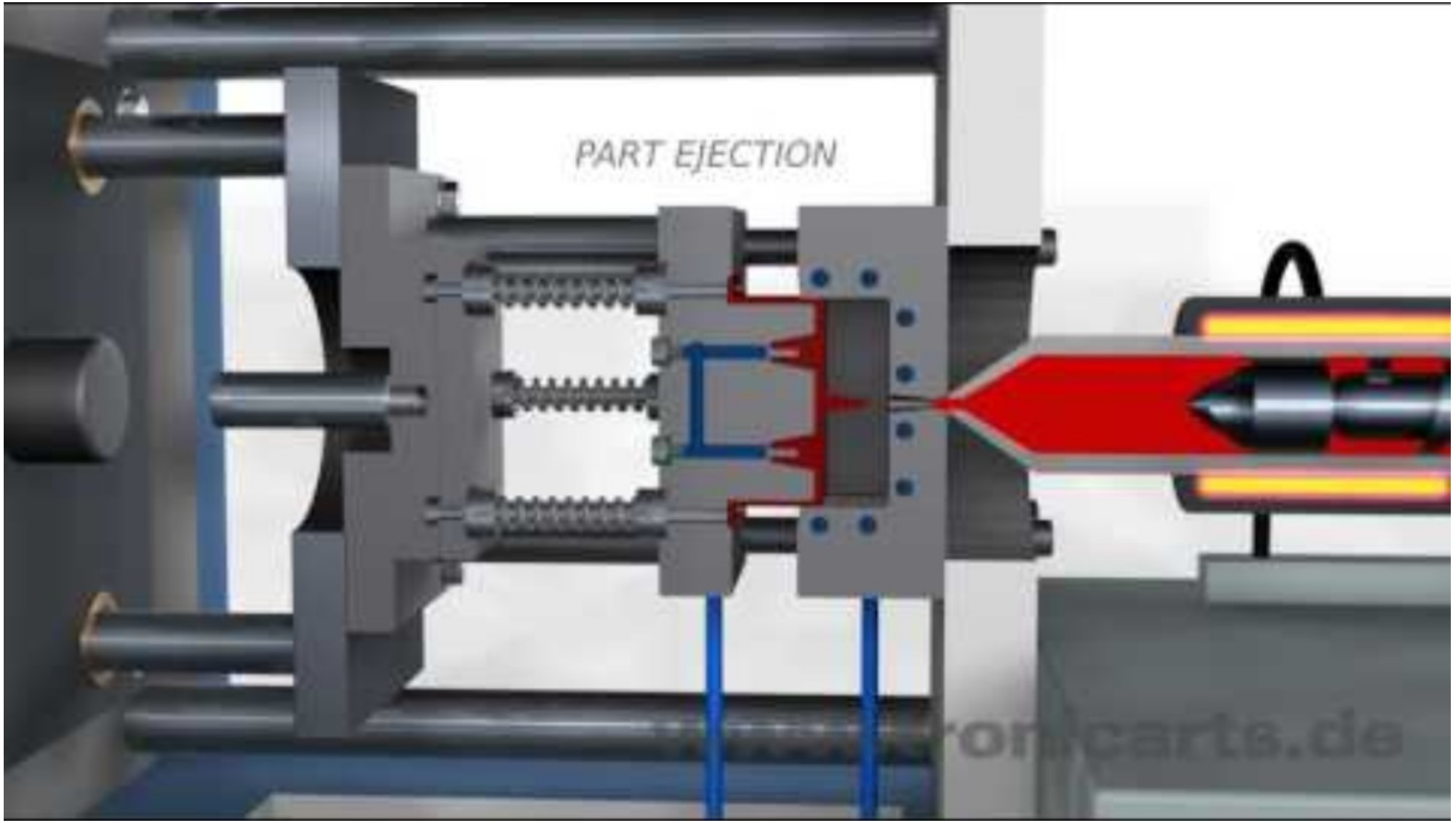
- 경기도에 위치한 H사는 자동차 부품 제조업체
- 사출성형기의 상태/조건에 따라서 제조 결과물의 정상/불량 여부를 판단하고자 함



<출처 : H사>



사출성형이란?



https://youtu.be/b1U9W4iNDiQ?si=HRAXk5kKv0iQ22F3

사출성형이란?



https://youtu.be/Opfj5Yppd3M?si=zSJGL9bE8MWwOBIQ

실습: 사출성형 제조공장 현장 문제 파악

- H사는 현재 사출성형기에 설정된 조건으로 제품을 생산해도 될지를 판단할 수 있는 AI 모델을 개발하고자 함

- 특성: 사출성형기에서 수집된 데이터 (사출샷 Lot 단위 PLC 데이터)
- 타겟: 불량 여부
  - 불량 -〉 1
  - 정상 -〉 0

데이터셋 스크립트

헤더		바디			
No	변수명	척도	유형 타입	변수 설명	속성/범주값
1	Lot no.	범주형	stirng	일정 단위로 생산된 사출성형물의 고유번호	5자리 문자열
2	금형상온도 (°C)	연속형	float64	금형 상판온도	범위 : 20~80°C, 소수점 8자리
3	금형하온도 (°C)	연속형	float64	금형 하판온도	범위 : 20~80°C, 소수점 8자리
4	냉각시간 (sec)	연속형	float64	금형에 원료 주입되어 사출물이 굳어지기까지 걸리는 시간	범위 : 20~40초, 소수점 8자리
5	노즐온도 (°C)	연속형	float64	사출기의 노즐 온도	범위 : 15~25°C, 소수점 8자리
6	냉각수 In 온도_Avg (°C)	연속형	float64	냉각수 유입온도 평균	범위 : 20~35°C, 소수점 8자리
7	냉각수 In 온도_Std	연속형	float64	냉각수 유입온도 표준편차	범위 : 0.1~3, 소수점 8자리
8	냉각수 Out 온도_Avg (°C)	연속형	float64	냉각수 출수온도 평균	범위 : 180~280°C, 소수점 8자리
9	냉각수 Out 온도_Std	연속형	float64	냉각수 출수온도 표준편차	범위 : 0.1~110, 소수점 8자리
10	보압속도 (mm/sec)	연속형	Float64	금형 내에 원료가 채워지도록 스크류를 전진시키는 속도	범위 : 0.5~6mm/s, 소수점 8자리
11	사출압력 (kg/cm^2)	연속형	float64	스크류가 보압 전환점에 도달했을 때 가해지는 압력	범위 : 10~210kg/cm^2, 소수점 8자리
12	사출시간 (sec)	연속형	float64	재료를 금형에 유입시키는 데 소요되는 시간	범위 : 1~6초, 소수점 8자리
13	실린더 온도 (°C)	연속형	float64	실린더 내부온도	범위 : 185~275°C, 소수점 8자리
14	작동유 온도 (°C)	연속형	float64	작동유 온도	범위 : 20~40°C, 소수점 8자리
15	쿠션량	연속형	float64	보압 완료 후 스크류의 위치	범위 : 0~10, 소수점 8자리
16	불량 여부	범주형	int64	사출품 품질 정상/불량 여부	0 : 정상, 1 : 불량

[참고] LOT란 무엇인가?

품목코드	특정 품목을 지칭하는 코드
LOT 번호	동일한 재료를 사용하여 제품 특성이 일정하다고 판단되는 품목들에게 공통적으로 주어지는 번호. 주로 같은 날짜에 생산되어진 제품별 묶음을 지칭합니다.
Serial번호	각 개별 단품들에게 주어지는 번호

품목코드와 LOT번호, Serial 번호와의 관계는 다음과 같습니다.

품목코드	LOT번호	Serial번호
1	N	
	1	N

LOT번호를 부여하는 일반적인 규칙은 품목코드 + 생산년월일 + 순번 이고, Serial번호는 LOT번호 + 순번이 입니다.

실습 5.
: 숫자 제조데이터 기반 AI 개발

(1) 데이터 탐색 및 전처리

(2) 로지스틱 회귀와 결정 트리

(3) 랜덤 포레스트

데이터 불러오기

- read_csv

```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/_for_exercise/Molding_Numeric_Data.csv', encoding='cp949')
```

- sample 또는 head

```
# 가독성 향상을 위해 소수점 출력 포맷을 소수점 넷째자리까지 나오도록 설정
pd.set_option('display.float_format', lambda x: '%.4f' % x)
```

```
df.sample(3)
```

	LOT NO.	금형상온도	금형하온도	냉각시간	노즐온도	냉각수In 온도_Avg	냉각수In 온도_Std	냉각수Out 온도_Avg	냉각수Out 온도_Std	보압속도	사출압력	사출시간	실린더온도	작동유온도	쿠션량	불량여부
28	22C06	58.3584	62.1516	28.1254	20.6531	27.1077	0.5433	267.5317	1.6551	4.1359	123.8713	3.5595	267.3783	38.0247	4.1117	0
58	22A08	21.6963	23.9684	29.2085	22.1934	28.5656	0.4489	265.4244	1.3444	4.5990	114.8477	1.7303	265.6115	28.6695	0.4839	1
3	22D01	27.9566	55.2658	25.8315	20.6531	27.1077	0.5433	267.4442	1.6877	4.8406	106.2104	4.7459	266.5327	38.4096	4.4406	0

데이터 불러오기

- info

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 16 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   LOT NO.        100 non-null    object
 1   금형상온도          100 non-null    float64
 2   금형하온도          100 non-null    float64
 3   냉각시간           100 non-null    float64
 4   노즐온도           100 non-null    float64
 5   냉각수In 온도_Avg   100 non-null    float64
 6   냉각수In 온도_Std   100 non-null    float64
 7   냉각수Out 온도_Avg  100 non-null    float64
 8   냉각수Out 온도_Std  100 non-null    float64
 9   보압속도           100 non-null    float64
 10  사출압력           100 non-null    float64
 11  사출시간           100 non-null    float64
 12  실린더온도          100 non-null    float64
 13  작동유온도          100 non-null    float64
 14  쿠션량            100 non-null    float64
 15  불량여부           100 non-null    int64
dtypes: float64(14), int64(1), object(1)
memory usage: 12.6+ KB
```

데이터 불러오기

- describe

	금형상온도	금형하온도	냉각시간	노즐온도	냉각수In 온도_Avg	냉각수In 온도_Std	냉각수Out 온도_Avg	냉각수Out 온도_Std	보압속도	사출압력	사출시간	실린더온도	작동유온도	쿠션량	불량여부
count	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000
mean	38.2261	44.8870	28.9078	20.7016	27.2411	0.5433	265.8295	3.4323	4.5507	106.8575	2.8003	266.4576	31.8462	2.8017	0.3100
std	12.2748	13.3766	1.8437	0.9266	1.1170	0.2766	8.3441	12.0531	0.5790	28.0154	1.2840	8.4616	4.9506	1.9133	0.4648
min	21.6275	21.9453	22.8005	16.2571	20.9357	0.2374	187.2452	0.1203	0.8591	13.5051	1.4962	188.3993	23.0494	0.1404	0.0000
25%	27.9918	41.8278	28.7591	20.6531	27.1077	0.5433	265.4268	1.3893	4.4472	96.3384	1.7653	265.3925	28.9650	0.9827	0.0000
50%	38.3051	41.8278	29.2085	20.6531	27.1077	0.5433	265.8376	1.5986	4.7637	113.0336	1.9892	267.2178	29.7226	2.4146	0.0000
75%	38.3051	58.1287	29.2085	20.6531	27.1077	0.5433	267.8664	1.7378	4.8308	121.7853	4.1145	269.4608	37.6349	4.4286	1.0000
max	59.6514	63.3119	35.2475	23.2689	30.2775	2.9689	270.9232	109.6399	5.6443	206.0861	4.8901	272.4080	39.5460	9.4090	1.0000

데이터 탐색하기

- 결측치 확인

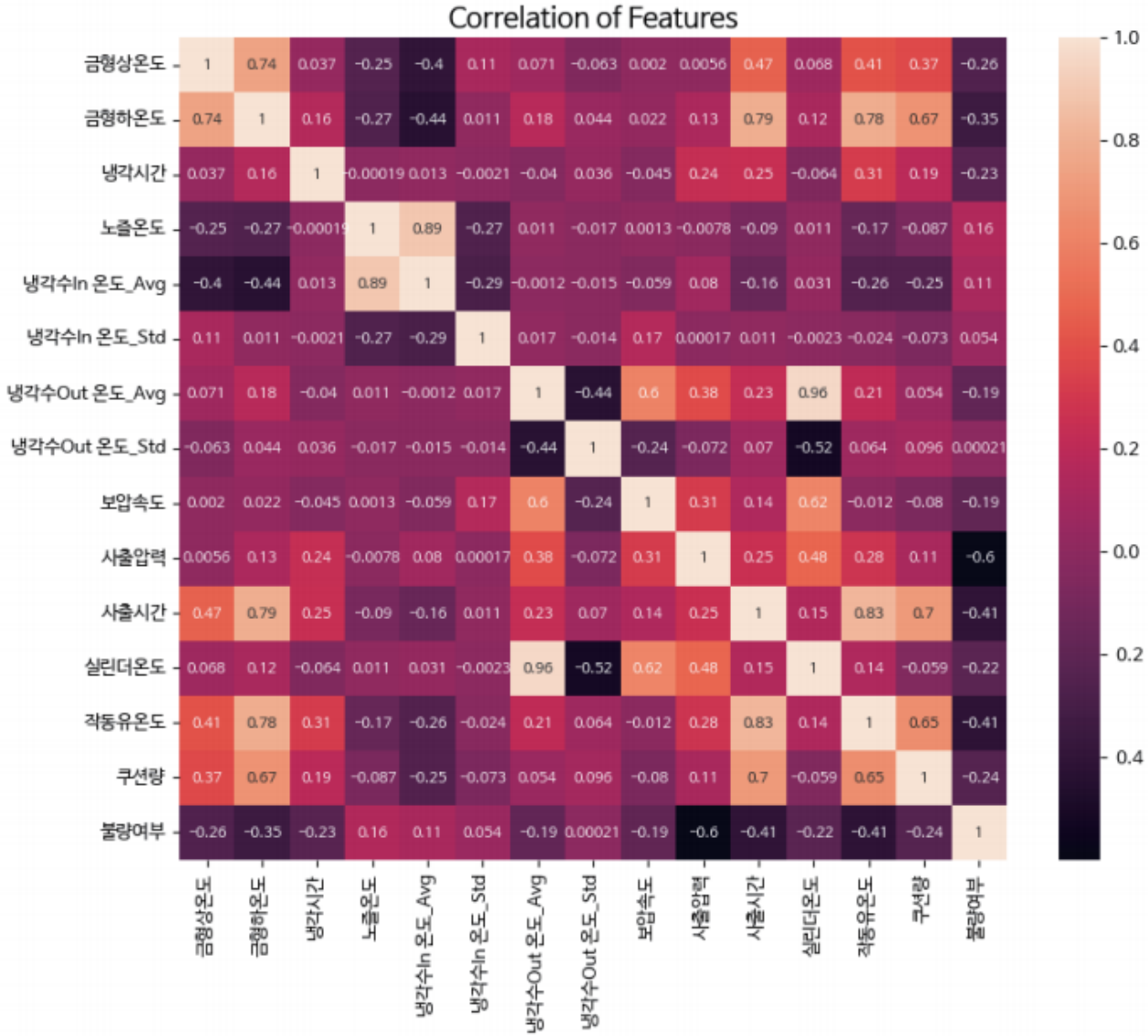
```
# 제조데이터 결측치 확인
df.isnull().sum()
```

```
LOT NO. 0
금형상온도 0
금형하온도 0
냉각시간 0
노즐온도 0
냉각수In 온도_Avg 0
냉각수In 온도_Std 0
냉각수Out 온도_Avg 0
냉각수Out 온도_Std 0
보압속도 0
사출압력 0
사출시간 0
실린더온도 0
작동유온도 0
쿠션량 0
불량 여부 0
dtype: int64
```

데이터 탐색하기

- 히트맵 그리기

교재 코드



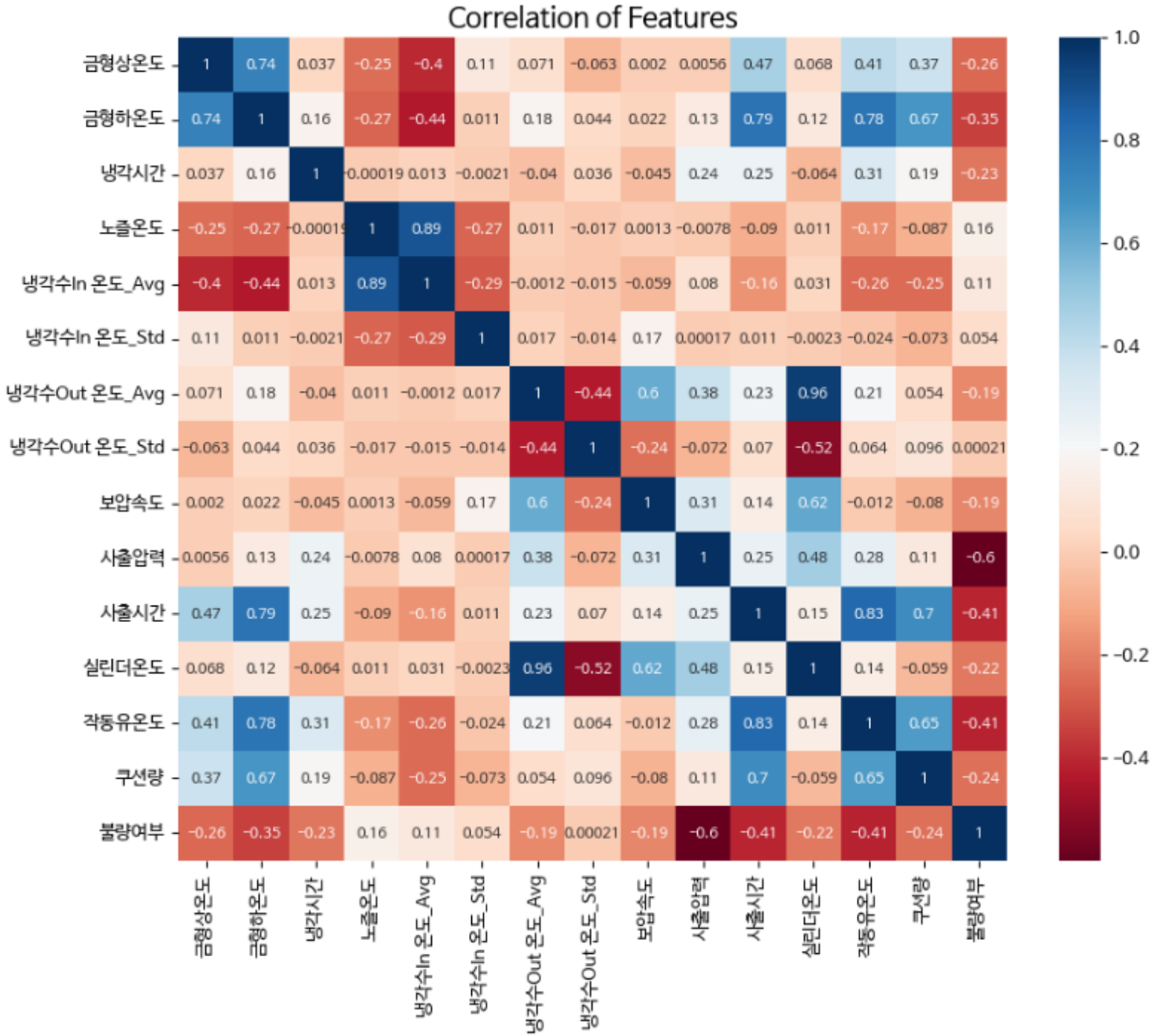
실습은 아래 코드로

```
import matplotlib.pyplot as plt
import seaborn as sns

heatmap_data = df.drop(['LOT NO.'], axis=1)

plt.figure(figsize=(10,8))
plt.title('Correlation of Features', size=15)

sns.heatmap(heatmap_data.corr(),
            cmap='RdBu',
            annot=True,
            annot_kws={'size':8})
plt.show()
```



데이터 분할 및 정규화

교재 코드

실습은 아래 코드로

```
from sklearn.model_selection import train_test_split

# 특성과 타겟 데이터 나누기
df_X = df.drop(['불량여부', 'LOT NO.'], axis=1 )
df_y = df['불량여부']

# 훈련데이터와 테스트 데이터 나누기
train_X, test_X, train_y, test_y = train_test_split(df_X, df_y, test_size=0.2, stratify=df_y)
```

```
from sklearn.preprocessing import StandardScaler

# 데이터 정규화
scaler = StandardScaler()
train_X_scaled = scaler.fit_transform(train_X)
```

실습 5.
: 숫자 제조데이터 기반 AI 개발

(1) 데이터 탐색 및 전처리

(2) 로지스틱 회귀와 결정 트리

(3) 랜덤 포레스트

실습 5.
: 숫자 제조데이터 기반 AI 개발

(1) 데이터 탐색 및 전처리

(2) 로지스틱 회귀와 결정 트리

(3) 랜덤 포레스트

앙상블(Ensemble) 학습

- 개념
  - **여러 개의 머신러닝 모델**을 만들어 그 **예측을 종합**하여 더 정확한 최종 예측을 만들어내는 기법

- 배깅 (Bagging = Bootstrap + Aggregating)
  - 앙상블 기법 중 하나

  - 동작 방식
    - 원본 학습 데이터에서 **무작위로 데이터를 복원 추출** → 여러 개의 서브 데이터셋
    - 각 모델은 이 **서로 다른 데이터셋으로 개별적으로 학습**
    - 최종적으로 모든 모델의 예측 결과를 **다수결 투표나 평균을 통해 종합**

  - 단일 모델이 특정 데이터에 과하게 학습되는 과적합을 방지하는 데 효과적

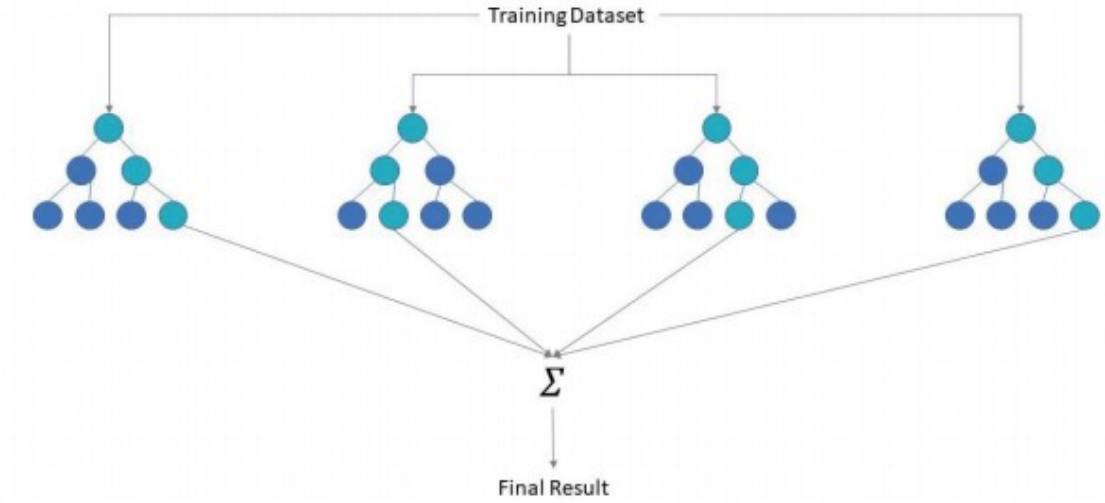
  - 대표 알고리즘: **랜덤 포레스트 (Random Forest)**

랜덤 포레스트(Random Forest)

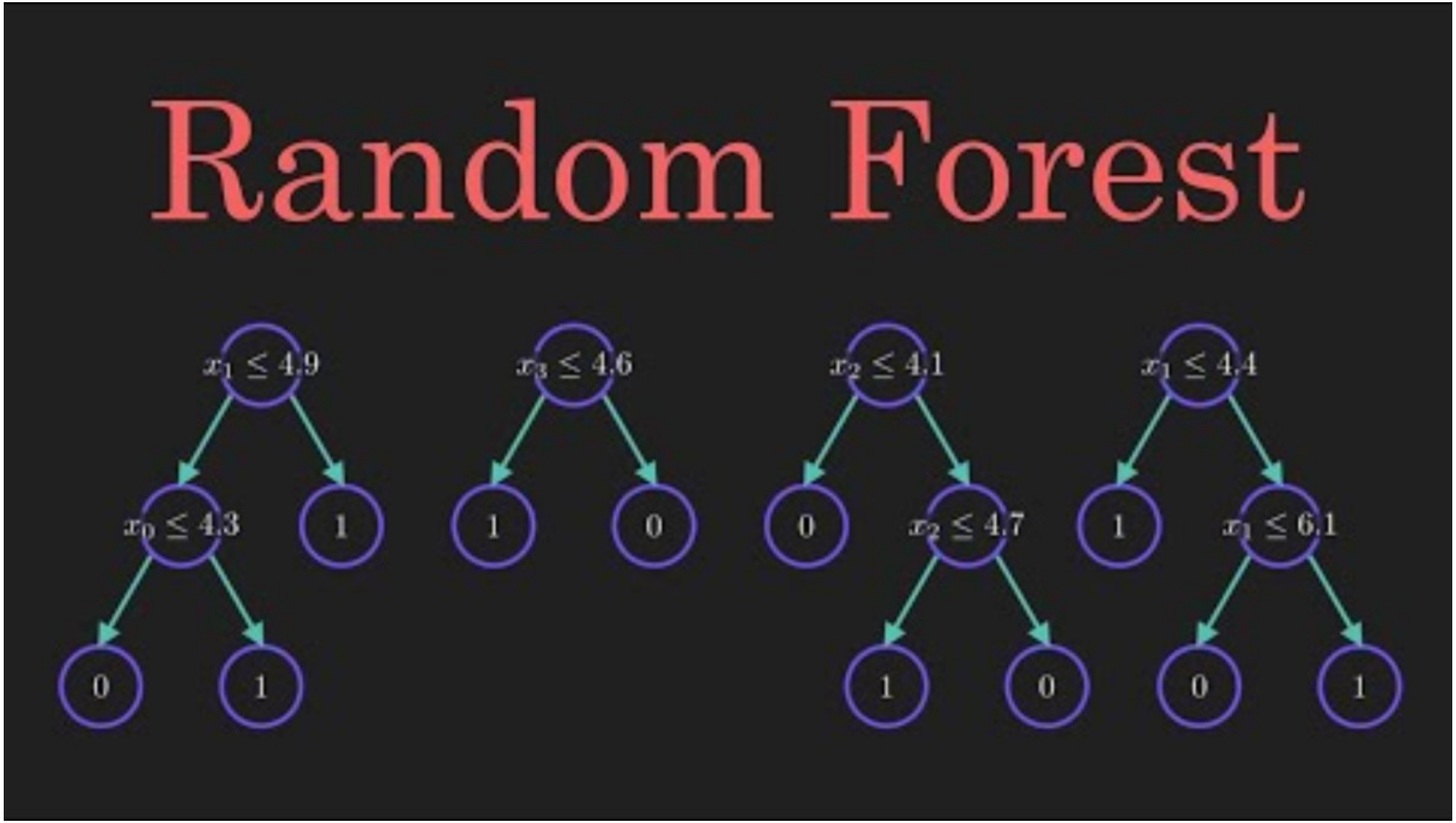
- 수많은 결정 트리를 모아 하나의 숲을 이루어 예측을 수행하는 앙상블 학습 방법

- 왜 "랜덤"이라고 부를까?
  - ① 데이터 무작위 추출
  - ② 특성 무작위 선택

- 왜 이렇게 할까?
  - ① 원래 학습 데이터에 덜 민감하게 반응
  - ② 결정 트리 간의 상관관계를 줄임



랜덤 포레스트(Random Forest)



https://youtu.be/v6VJ2RO66Ag?si=u5i5m_8CkABNsV_K

렌덤 포레스트(Random Forest)

장점	단점
높은 정확도 : 일반적으로 단일 모델에 비해 매우 높은 예측 성능	**해석의 어려움** : 수많은 트리가 얽혀 있어, 모델이 왜 그런 예측을 했는지 직관적으로 이해하기 어려움
과적합(Overfitting) 방지 : 여러 트리의 결과를 평균내므로, 특정 데이터에 과하게 학습되는 경향이 적음	**메모리 및 계산 비용** : 많은 트리를 생성하고 저장해야 하므로, 메모리 소모가 크고 예측 속도가 느릴 수 있음
데이터 전처리 용이 : 데이터의 정규화나 스케일링에 크게 영향을 받지 않음	**매개변수 튜닝 필요** : 최적의 성능을 위해 트리의 개수, 최대 깊이 등 여러 매개변수를 조정해야 함
특성 중요도 제공 : 어떤 특성이 예측에 더 큰 영향을 미쳤는지 알 수 있어, 변수 선택에 유용	
결측치(Missing Value) 처리 : 데이터에 결측치가 있어도 비교적 잘 동작함	

모델 학습 및 성능지표 계산

교재 코드

실습은 아래 코드로

```
from sklearn.ensemble import RandomForestClassifier

#rf = RandomForestClassifier(random_state=42)
rf = RandomForestClassifier(n_estimators=10, max_depth=5, random_state=42)

rf.fit(train_X_scaled, train_y)
```

```
# 성능지표
# 정확도: sklearn.metrics 함수 사용
from sklearn.metrics import accuracy_score

# 입력 데이터를 훈련 데이터에 맞춰서 표준화해야 함
test_X_scaled = scaler.transform(test_X)


test_pred = rf.predict(test_X_scaled)
train_pred = rf.predict(train_X_scaled)

test_acc = accuracy_score(test_y, test_pred)
train_acc = accuracy_score(train_y, train_pred)
print(f"test_accuracy: {test_acc}")
print(f"train_accuracy: {train_acc}")
```

```
test_accuracy: 0.85
train_accuracy: 0.975
```

새로운 데이터에 대한 예측값

새로운 제조데이터 준비

```
df.columns
```

```
Index(['LOT NO.', '금형상온도', '금형하온도', '냉각시간', '노즐온도', '냉각수In 온도_Avg',
       '냉각수In 온도_Std', '냉각수Out 온도_Avg', '냉각수Out 온도_Std', '보압속도', '사출압력',
       '사출시간', '실린더온도', '작동유온도', '쿠션량', '불량여부'],
      dtype='object')
```

```
test_data = {'금형상온도': [58, 23],
             '금형하온도': [62, 24],
             '냉각시간': [29,29],
             '노즐온도': [20,23],
             '냉각수In 온도_Avg':[27,29],
             '냉각수In 온도_Std':[0.54,0.36],
             '냉각수Out 온도_Avg':[267,265],
             '냉각수Out 온도_Std':[1.5,0.8],
             '보압속도':[4.8,4.9],
             '사출압력':[105,120],
             '사출시간':[4.8,2.1],
             '실린더온도':[268,263],
             '작동유온도':[36,26],
             '쿠션량':[3.5,0.5] }

test_data = pd.DataFrame(test_data)
```

새로운 데이터에 대한 예측값

새로운 제조데이터 스케일링 + 예측

```
test_data_scaled = scaler.transform(test_data)
```

```
predicted_result = rf.predict(test_data_scaled)

for result in predicted_result :
    if result == 1 :
        print("불량")
    else :
        print("양품")
```

```
양품
불량
```

금형상온도	금형하온도	냉각시간	노출온도	냉각수1h온도Ag	냉각수1h온도Cu	냉각수2t온도Ag	냉각수2t온도Cu	보압속도	사출압력	사출시간	실린더온도	작동유온도	쿠션량	불량여부
58	62	29	20	27	0.54	267	1.5	4.8	105	4.8	268	36	3.5	양품
23	24	29	23	29	0.36	265	0.8	4.9	120	2.1	263	26	0.5	불량

특성 중요도 확인

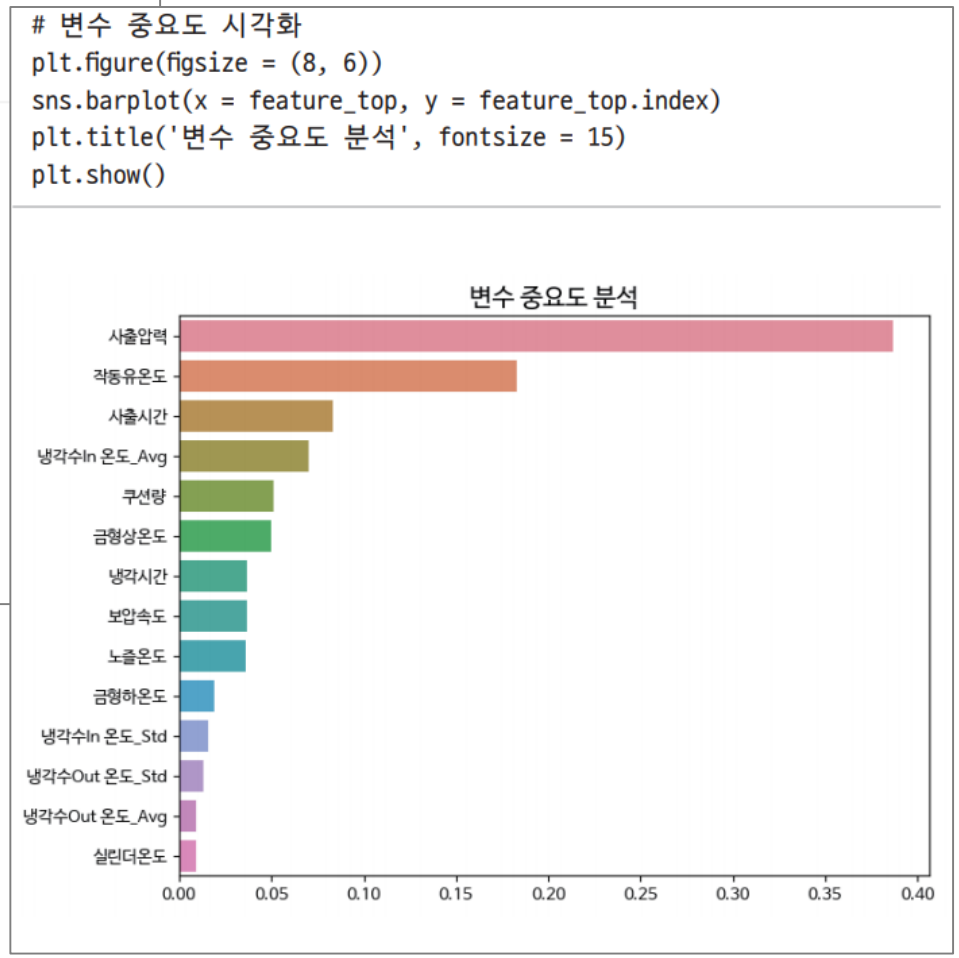
특성 중요도

```
feature_importances = pd.Series(rf.feature_importances_.round(3), index=train_X.columns)
feature_importances_sorted = feature_importances.sort_values(ascending=False)

print(feature_importances_sorted)
```

```
사출압력             0.3620
작동유온도            0.1360
사출시간             0.1060
냉각수Out 온도_Std    0.0850
냉각수In 온도_Avg     0.0620
냉각수Out 온도_Avg    0.0510
금형하온도            0.0450
보압속도             0.0360
실린더온도            0.0300
금형상온도            0.0300
노즐온도             0.0250
쿠션량              0.0190
냉각수In 온도_Std     0.0080
냉각시간             0.0050
dtype: float64
```

```
# 변수 중요도 시각화
plt.figure(figsize = (8, 6))
sns.barplot(x = feature_top, y = feature_top.index)
plt.title('변수 중요도 분석', fontsize = 15)
plt.show()
```



1. 다음 중 랜덤 포레스트(Random Forest)에 대한 설명으로 옳지 않은 것은 무엇인가요?

① 대표적인 앙상블(ensemble) 학습법이다.
② 파이썬 라이브러리 scikit-learn을 사용하면 쉽게 구현할 수 있다.
③ sklearn.ensemble 모듈에서 RandomForestClassifier를 사용한다.
④ 숲을 만들 때 나무의 개수를 n_estimators 파라미터로 지정하면 오류가 발생하기에 주의해야 한다.
⑤ 모델 구성 시 데이터 중복을 허용하기 때문에 단일 데이터가 여러 번 선택될 수도 있다.

2. 파이썬에서 CSV 파일을 읽기 위한 판다스(Pandas) 라이브러리의 함수는 무엇인가요?

① info()
② read_csv()
③ index()
④ pd.DataFrame()
⑤ describe()

3. 그래프를 그리기 위해서 Matplotlib 라이브러리를 사용합니다. 다음 중 그래프를 화면에 나타나도록 하는 matplotlib.pyplot 모듈의 함수는 무엇일까요?

① plot()
② axis()
③ title()
④ show()

참고 자료

- 『제조AI빅데이터 분석 기법 (코딩편)』, 김일중 외 6인 공저, 보민출판사 (2024)

끝!